

尚硅谷嵌入式技术之四轴无人机_飞控板

(作者：尚硅谷研究院)

版本：V1.1.0

第 1 章 项目需求

本项目是基于 STM32 的微型四轴无人机，相较于中大型四轴无人机，微型四轴无人机具有**成本低**、**事故代价低**、结构简单和量产率高等优势。控制核心采用 STM32F103C8T6，姿态运动传感器选择 **MPU6050**。无人机通过 **Si24R1** (NRF24L01) 与遥控器进行 2.4G 无线通信，实现了即时有效地接遥控器指令，通过串级 PID 进行姿态控制，从而在空间中实现自由移动。

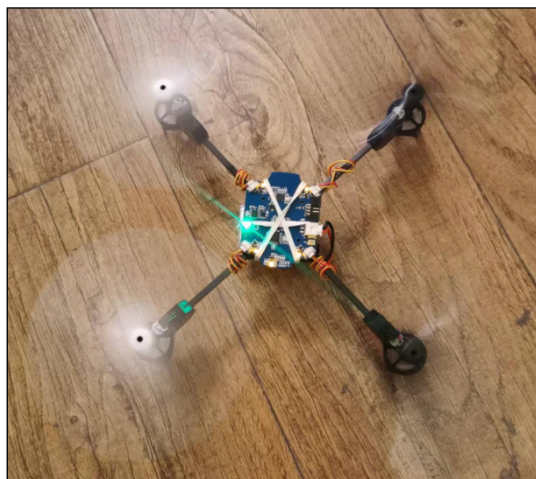
第 2 章 四轴无人机背景知识

四旋翼飞行器 (quadrotor aircraft) 又称四轴直升机 (quadcopter) 或简称四转子 (quad)，是用**大小相同**、**两侧对称**分布的**四个旋翼**产生升力的多轴飞行器，属于直升机的一种。

这是大疆的一台正在飞行的四轴无人机.人民币:x 万元



这是咱们要自己编程控制**尚硅谷四轴无人机**. 人民币:x 百元



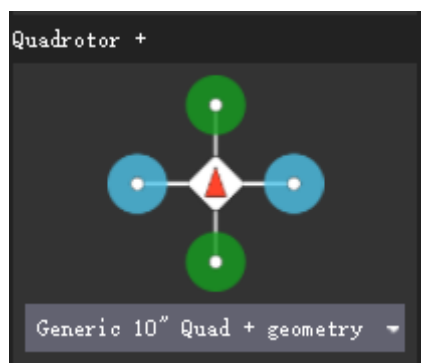
虽然他们价格相差百倍,但是基本控制原理是相同的,从成本考虑,尚硅谷四轴无人机才是你的首选.

2.1 四轴无人机结构

四轴飞行器的结构可以归结为 2 类 :X 型和+字形, 其中尤一 X 项最为常见,因为它控制比较简单, 稳定性号,尤其适合初学者.

2.1.1 十字形结构

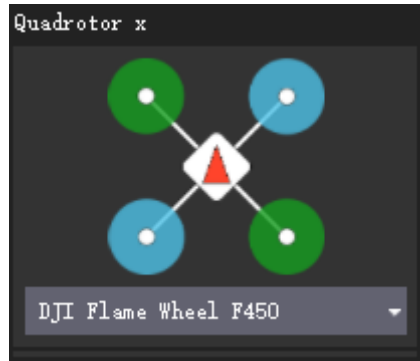
典型特征是机头方向对着 1 个旋翼.



2.1.2 X 字形结构

典型特征是机头处于 2 个旋翼之间.

我们尚硅谷使用的就属于这种 X 字型结构的无人机.



2.2 一些常用术语

(1) 飞控

飞控指飞行器的电子控制部分，硬件包括传感器部分的惯性导航模块和控制部分的 MCU。软件主要包括控制算法等。

(2) pitch, yaw, roll

指三维空间中飞行器的旋转状态，对应中文分别是俯仰，偏航，横滚。

(3) 惯性导航模块

简称 **IMU**(Inertial Measurement Unit)，陀螺仪传感器和加速度传感器提供的三轴运动数据模块。

(4) 地理坐标系

指地球所在的坐标系，这个坐标系是固定不变的，正北，正东，正上方分别表示 X，Y，Z 轴。

(5) 姿态解算

英文 attitude algorithm，也叫做姿态融合。姿态解算是指把陀螺仪、加速度计、罗盘等的的数据融合在一起，得出飞行器的空中姿态。

(6) 四元数

四元数是一种扩展复数的数学结构，用于表示**三维空间中的旋转**。

(7) 欧拉角

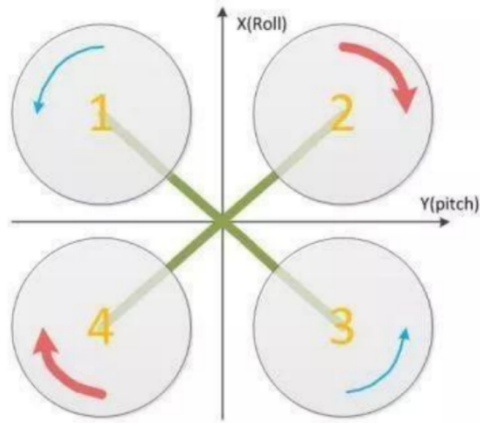
欧拉角是一种用三个角度描述物体在三维空间中旋转的方法。这三个角度分别描述物体绕固定坐标系的三个轴的旋转。可以根据四元数计算出来欧拉角。

(8) 空心杯电机

一种特殊类型的直流无刷电机，因其转子采用空心杯设计而得名。与传统的有铁芯电机相比，空心杯电机在结构、性能和应用上都有显著的特点和优势。

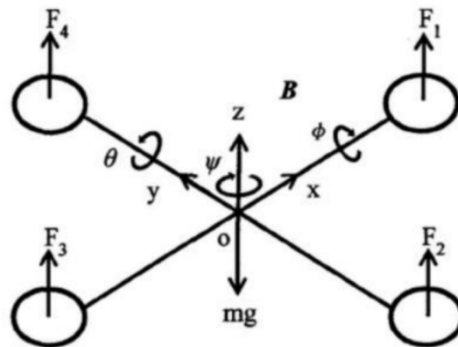
2.3 飞行动力学原理

我们的四轴飞行器是 X 型的，其稳定性大于十字型的四轴飞行器，它的特征是，机头夹在两个电机之间。



电机 1 和电机 3 逆时针旋转的同时，电机 2 和电机 4 顺时针旋转，因此当飞行器平衡飞行时，**陀螺效应**和**空气动力扭矩效应**均被抵消。

2.3.1 垂直运动



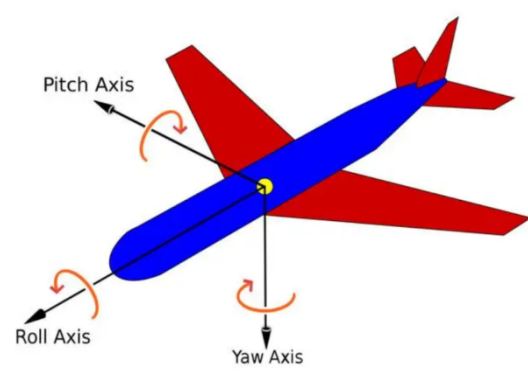
同时增加四个电机的输出功率，旋翼转速增加使得总的拉力增大，当总拉力足以克服整机的重量时，四旋翼飞行器便离地垂直上升；反之，同时减小四个电机的输出功率，四旋翼飞行器则垂直下降，直至平衡落地，实现了沿 z 轴的垂直运动。当外界扰动量为零时，在旋翼产生的升力等于飞行器的自重时，飞行器便保持悬停或匀速飞行状态。

当 $mg > F_1 + F_2 + F_3 + F_4$ ，此时做下降加速飞行

当 $mg < F_1 + F_2 + F_3 + F_4$ ，此时做升高加速飞行

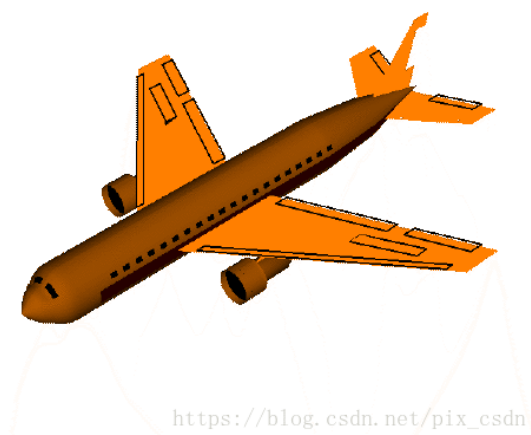
当 $mg = F_1 + F_2 + F_3 + F_4$ 此时垂直上保持匀速飞行或者悬停

2.3.2 水平运动



2.3.2.1 偏航运动

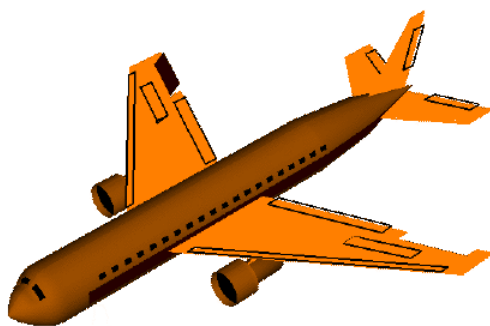
正前方为机头，此时以机头为参考做旋转运动。在我们的四轴无人机中， ω_4 和 ω_2 的转向与 ω_1 和 ω_3 的转向相反，这是因为螺旋桨旋转会产生水平扭矩。于是当 $\omega_4 + \omega_2 \neq \omega_1 + \omega_3$ 就会产生水平旋转，这就是偏航原理。至于是顺旋还是逆旋，要根据实物电机正反转搭配正反桨来决定。



2.3.2.2 横滚运动

当 $F_4 + F_3 > F_1 + F_2$ 向右飞行

当 $F_4 + F_3 < F_1 + F_2$ 向左飞行

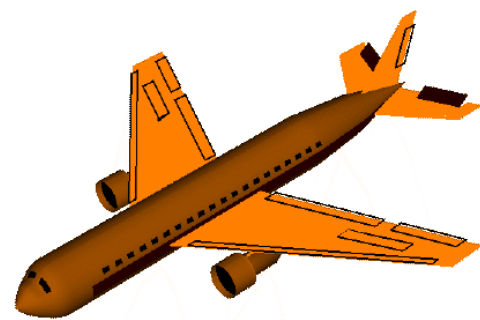


https://blog.csdn.net/pix_csdn

2.3.2.3 俯仰运动

当 $F1+F4 < F2+F3$ 向前飞行

当 $F1+F4 > F2+F3$ 向后飞行



https://blog.csdn.net/pix_csdn

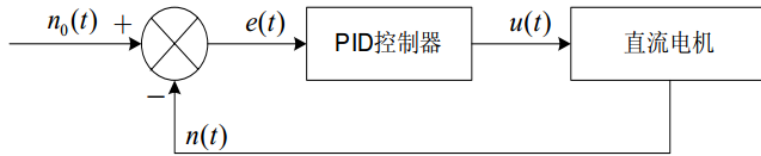
2.4 PID 控制系统

将偏差的比例（Proportion）、积分（Integral）和微分（Differential）通过线性组合构成控制量，用这一控制量对被控对象进行控制，这样的控制器称 **PID 控制器**。

PID 控制应该算是应用非常广泛的控制算法了。小到控制一个元件的温度，大到控制无人机的飞行姿态和飞行速度等等，都可以使用 PID 控制。

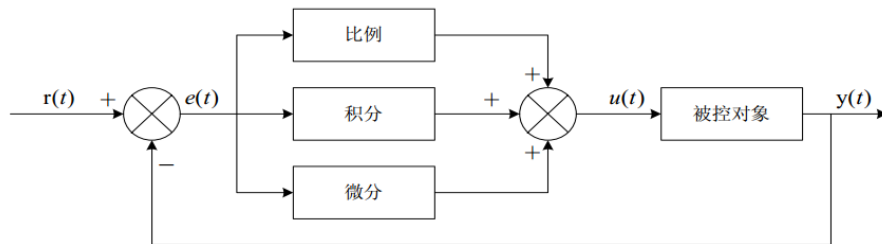
2.4.1 模拟 PID 控制

在模拟控制系统中，控制器最常用的控制规律是 PID 控制。假设要控制一个直流电机，下面就是它的控制框图。



在上图中, 实际转速 $n(t)$ 与期望转速 $n_0(t)$ 进行比较, 其差值 $e(t) = n(t) - n_0(t)$ 一般称为偏差(误差), 传递给 PID 控制器后得到 $u(t)$, 然后驱动直流电机来改变其转速, 从而可以让实际转速与期望转速越来越接近, 直至相等。

如果换成更通用的控制对象, 可以把框图抽象成如下



其中比例部分就是 PID 中的 P, 积分就是 PID 中的 I, 微分就是 PID 中的 D.

图中 $r(t)$, 是期望值, $y(t)$ 是系统的实际输出值, $e(t)$ 给定值与实际输出值构成控制偏差: $e(t) = y(t) - r(t)$

所以模拟 PID 控制器的控制规律为:

$$u(t) = K_p * e(t) + K_i * \int e(t) dt + K_d * \frac{de(t)}{dt}$$

其中: K_p 为控制器的比例系数, K_i 为控制器的积分系数, K_d 为控制器的微分系数.

2.4.2 数字 PID 控制

由于计算机的出现, 计算机进入了控制领域。人们也将 PID 控制理论引入到了计算机控制领域。

由于计算机控制是一种**采样控制**, 它只能根据采样时刻的偏差计算控制量, 而不能像模拟控制那样连续输出控制量, 进行连续控制。

由于这一特点, 上述公式中的积分项和微分项不能直接使用, 必须进行**离散化**处理。

假设采样周期为 T , e_k 表示第 k 次采样的偏差, u_k , 则会由如下的离散化的采样公式

$$u_k = K_p * e_k + K_i * \sum_{j=0}^k e_j T + K_d * \frac{e_k - e_{k-1}}{T}$$

或者

$$u_k = K_p * e_k + K_i * (e_0 + e_1 + \dots + e_{k+1} + e_k) * T + K_d * \frac{e_k - e_{k-1}}{T}$$

只要其中的**采样周期足够小**，离散后的结果与连续过程就十分接近了。

2.4.3 采样周期的选择

前面说到采样周期要足够小，那么小到什么程度才叫无限小呢？

根据香农（Shannon）采样定律：为不失真地复现信号的变化，采样频率至少应大于或等于连续信号**最高频率分量的 2 倍**。

采样周期的选择，通常按照过程特性与干扰大小适当来选取采样周期：

- （1）对于响应快、（如流量、压力）波动大、易受干扰的过程，应选取较短的采样周期；
- （2）反之，当过程响应慢（如温度）、滞后大时，可选取较长的采样周期。

2.4.4 串级 PID

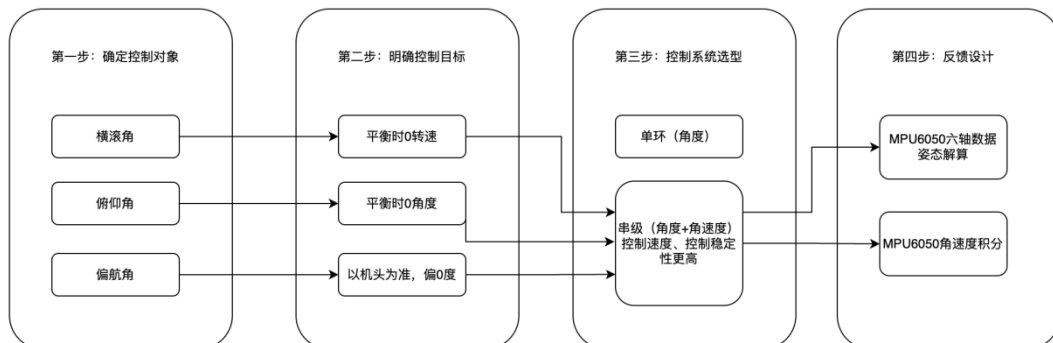
如果把 A PID 控制系统的输出，作为另外一个 B PID 控制系统的期望值(输入)，则称为**串级 PID**。其中 A PID 称为外环, B PID 称为内环

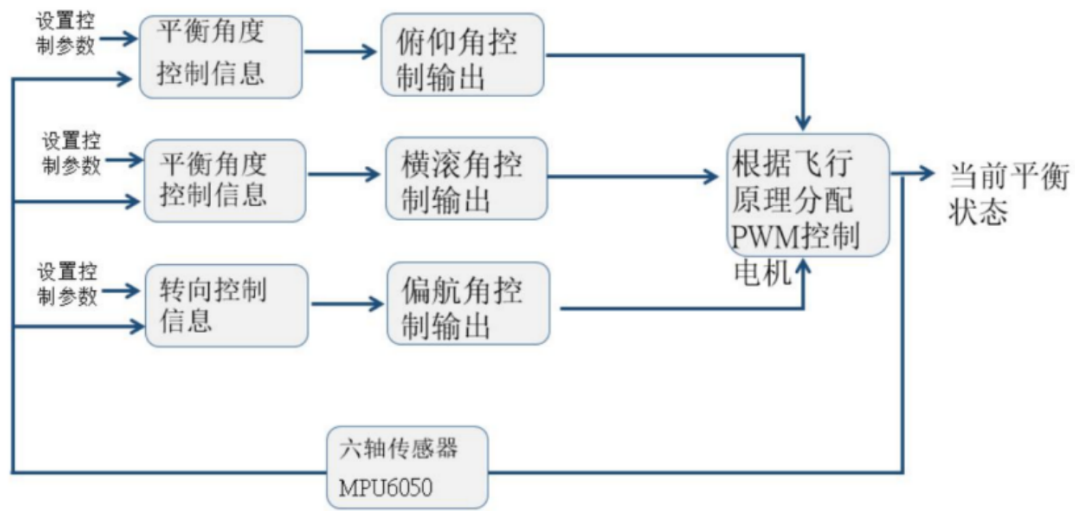
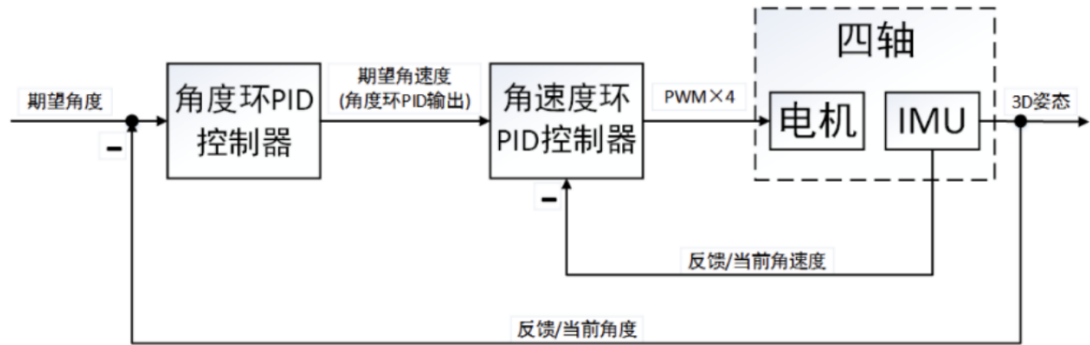
与之对应的单个 A PID 或单个 B PID 称为**单环 PID**

串级 PID 相比单环 PID，更能抑制干扰。

串级 PID 设计的考虑如下：

- （1）计算角度时要用到角速度和加速度，由于加速度容易受到震动的干扰，导致角度也容易收到干扰。所以把角度放外环，角速度放内环增加稳定性
- （2）角度要通过加速度和角速度的解算，所以**角速度的反应速度快于角度**，把**角速度放在内环**，可以增加系统的**反应速度**





2.5 常用姿态解算方式

方法	描述	优点	缺点
角速度积分（欧拉角）	类似二维平面上 $\angle \theta = \omega * t$ 。利用三维旋转矩阵映射到三维时空。	原理和运算量都简单，稳定性好	长时间积分会出现积分偏差。欧拉角存在万向锁的问题。
加速度解算	类似二维平面反三角函数 $\angle \theta = \arctan(a \text{ 对边} / b \text{ 临边})$ 。三维旋转矩阵映射到三维时空	原理简单好理解	容易受振动影响
互补解算	由于单一传感器获得角度都有缺点，那就综合上述二者	综合了上述两者的优点	会受振动影响，并且动态性能差，大角度变化反应慢
四元数姿态解算	通过某个固定的算法得到角度	算法固定，数学理论成熟，解算准确，计	会受振动影响，角度回复有延迟。算法不容易

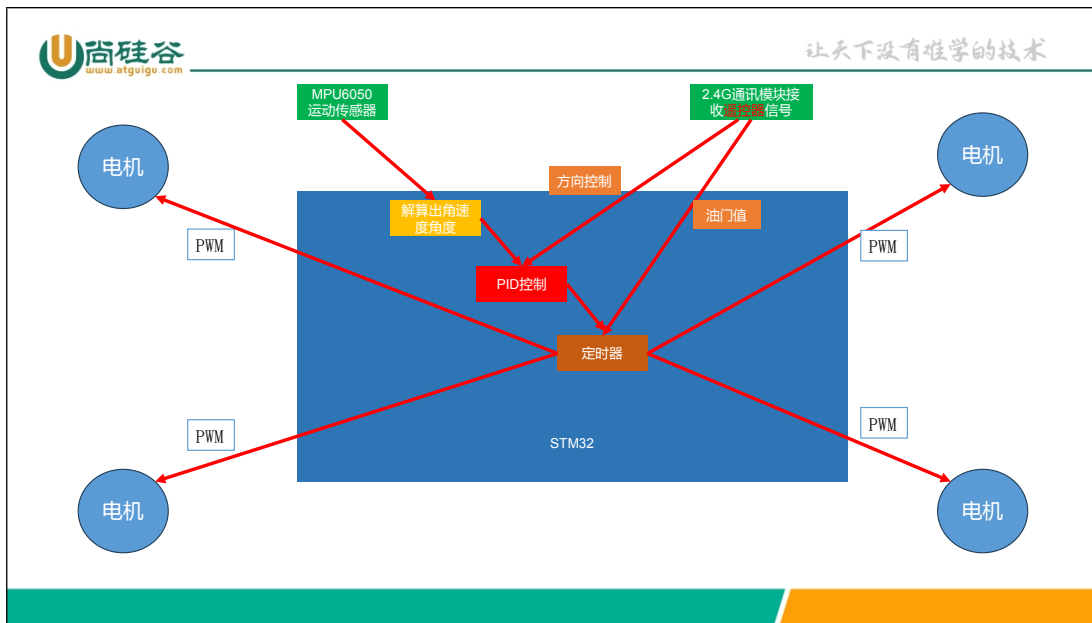
		算量小，效果直逼卡 尔曼	理解
卡尔曼姿态评估	所谓评估就是综合某些因素，去估计姿态，就如同一部车驶来，你会根据它的速度评估下一秒的位置。	动态性能好	模型复杂，计算量大，理论复杂。参数 难调

四元数解算算法的数学推导原理，一般属于研究生研究的范畴，建议一般学习不要去深入研究，只需要明白它的作用就好。虽然 MPU6050 的 DMP 库可以帮助我们得到四元数，但是 MPU6050 精度和稳定性都没有自己读取 MPU6050 做四元数姿态来好，所以本次采用的读取 MPU6050 的加速度和角速度做四元数算法解算姿态，最终得到**欧拉角**。

欧拉角和**角速度**就是我们在进行姿态控制时要控制的对象。

第 3 章 硬件架构

3.1 系统总体设计



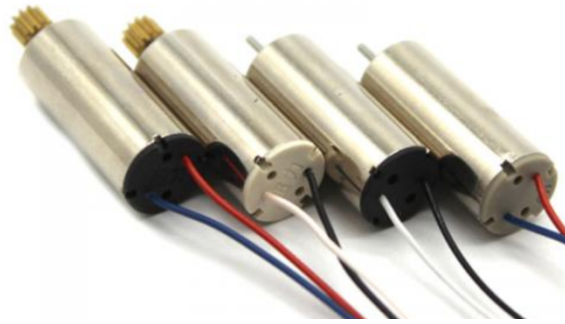
3.2 硬件选型

3.2.1 主控芯片

使用的是 ST 公司的 **STM32F103C8T6**。主频最高可达 72MHz, SRAM 有 20K, Flash 有 64K。定时器有 4 个。

3.2.2 电机

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)



我们使用的是 8520 空心杯电机. 主要用于单桨遥控飞机和四轴飞行器, 具有扭力大重量轻, 飞行时间长等优点. 由顺时针旋转和逆时针旋转 2 种.(注意: 一种空心杯只能有一种转动方向)

尺寸: 8.5mm 直径, 20mm 长度.

工作电压: 通常为 3.7V (单节锂电池)

无负载转速: 在额定电压下, 通常为几万转每分钟 (RPM)

无负载电流: 通常在几十毫安 (mA) 左右.

重量: 几克, 通常不超过 10 克.

通常用于驱动小型四旋翼无人机的螺旋桨, 提供高转速和高功率密度。

8520 空心杯电机因其高效、轻便、快速响应等特点, 广泛应用于各种微型设备中。其空心杯设计使得电机在高性能需求的微型应用中表现出色。尽管其制造成本较高, 但在需要高性能和精密控制的场景下, 其优势是无可替代的。

使用 PWM 可以方便的驱动电机转动

3.2.3 运动传感器

我们选用的是非常经典的 MPU6050 运动传感器。

包含一个 **3 轴陀螺仪** 和一个 **3 轴加速度计**, 并集成了一个数字运动处理器 (DMP)。这种传感器广泛应用于无人机、机器人、智能手机和可穿戴设备中, 用于姿态检测和运动跟踪。

MPU605x 加速度计通过测量施加在传感器上的加速度来确定物体的运动状态, 而陀螺仪通过测量角速度来检测旋转运动。

传感器的数据可以通过 **I2C** 接口读取。

MPU6050 是一种功能强大的 6 轴运动传感器, 广泛应用于姿态控制和运动检测。其内置的 DMP 使得复杂的运动算法处理更加高效, 适合多种嵌入式应用。

另外,需要注意的是,内置的 DMP 处理的数据不够准确,我们是[直接读取原始数据,在外面经过 MCU 来进行四元素和欧拉角的运算](#),虽然增加了 MCU 的运算负载,但是运算结果更精确了.



MPU6050介绍.do
cx

3.2.4 激光测距传感器

3.2.5 2.4G 通讯

遥控器是通过 2.4G 通讯实现的远程控制无人机的飞行.

我们使用的是国产 2.4G 无线通讯芯片 Si24R1.

Si24R1 是由中国公司设计和制造的射频芯片,其设计灵感和功能类似于 Nordic Semiconductor 的 nRF24L01+ 芯片。Si24R1 具有高度集成、低功耗和高数据传输速率的特点。

工作频率: 2.4GHz ISM 频段. 数据传输速率: 最高可达 2Mbps. 支持自动重传和自动确认. 多达 126 数据信道. 低成本、高性能: 适用于大规模应用. 传输距离: 视环境和天线设计,通常在几十米到数百米范围内.

Si24R1 通过 SPI 接口与微控制器通信。



Si24R1_数据手册_
V1.3.pdf

Si24R1 是一款高性能、低成本的 2.4GHz 射频收发芯片,适用于多种无线通信应用。它的低功耗、高数据速率和多通道支持,使其成为无线传感器网络、智能家居、遥控玩具和计算机外设等领域的理想选择。通过 SPI 接口,可以方便地与各种微控制器进行通信,实现无线数据传输。

第 4 章 软件架构



第 5 章 FreeRTOS 的移植



移植.docx

第 6 章 公共层

6.1 公共配置模块

6.1.1 Com_Types.h

```

#ifndef __COM_TYPES_H__
#define __COM_TYPES_H__
#include "gpio.h"

// LED 结构体
typedef struct
{
    GPIO_TypeDef *port;
    uint16_t pin;
    uint32_t last_tick;
} LED_Struct;
    
```

```
// 遥控状态

typedef enum {

    DISCONNECTED,

    CONNECTED

} Remote_Status;


// 飞行状态

typedef enum {

    IDEL,

    NORMAL,

    FIX_HEIGHT,

    FAULT

} Flight_Status;


// 电机结构体

typedef struct

{

    TIM_HandleTypeDef *htim;

    uint32_t channel;

    int16_t speed;

} Motor_Struct;


// 通用类型

typedef enum {

    COM_OK,

    COM_ERROR

} Com_Status;


// 遥控数据结构体

typedef struct {

    int16_t THR;        // 油门

    int16_t YAW;        // 偏航

    int16_t PIT;        // 俯仰

    int16_t ROL;        // 横滚
```

```
uint8_t fix_height; // 1:定高/取消定高 0:保持当前状态

uint8_t power_down; // 1:关机 0:保持当前状态
} Remote_Struct;

// 油门状态
typedef enum {
    FREE,
    MAX,
    LEAVE_MAX,
    MIN,
    UNLOCK
} THR_Status;

// 陀螺仪结构体
typedef struct
{
    int16_t x;
    int16_t y;
    int16_t z;
} Gyro_Struct;

// 加速度结构体
typedef struct
{
    int16_t x;
    int16_t y;
    int16_t z;
} Accel_Struct;

// 六轴结构体
typedef struct
{
    Gyro_Struct gyro;
    Accel_Struct accel;
```

```
} GyroAccel_Struct;

// 欧拉角
typedef struct
{
    float pitch;
    float roll;
    float yaw;
} EulerAngle_Struct;

#endif
```

6.2 调试模块

6.2.1 Com_Debug.h

```
#ifndef __COM_DEBUG_H
#define __COM_DEBUG_H

#include "usart.h"
#include "stdio.h"
#include "stdarg.h"
#include "string.h"

/* 开发调试项目 => 需要使用大量的 printf 打印信息 */
/* 项目运行时候 => 不再需要打印信息 */

#define DEBUG_ENABLE

/* 如果定义了 DEBUG_ENABLE 表示开启打印 */

#ifdef DEBUG_ENABLE

#define _FILE_NAME (strchr(__FILE__, '/') ? strchr(__FILE__, '/') + 1 : __FILE__)
#define FILE_NAME (strchr(_FILE_NAME, '\\') ? strchr(_FILE_NAME, '\\') + 1 :
_FILE_NAME)
```



```
#define debug_init() Debug_Init()

#define debug_printf(format, ...) printf("[%s:%d]" format, FILE_NAME, __LINE__,
##__VA_ARGS__)

#define debug_printfln(format, ...) printf("[%s:%d]" format "\r\n", FILE_NAME,
__LINE__, ##__VA_ARGS__)

/* 没有定义 DEBUG_ENABLE 表示关闭打印 */

#else

#define debug_init()

#define debug_printf(...)

#define debug_printfln(format, ...)

#endif

void Debug_Init(void);

#endif
```

6.3 滤波模块

6.3.1 Com_Filter.h

```
#ifndef __COM_FILTER_H__
#define __COM_FILTER_H__
#include "stdint.h"

/* 卡尔曼滤波器结构体 */
typedef struct
{
    float LastP; // 上一时刻的状态方差（或协方差）
    float Now_P; // 当前时刻的状态方差（或协方差）
    float out;    // 滤波器的输出值，即估计的状态
    float Kg;     // 卡尔曼增益，用于调节预测值和测量值之间的权重
    float Q;      // 过程噪声的方差，反映系统模型的不确定性
    float R;      // 测量噪声的方差，反映测量过程的不确定性
} KalmanFilter_Struct;
```

```
extern KalmanFilter_Struct kfs[3];

int16_t Com_Filter_LowPass(int16_t newValue, int16_t preFilteredValue);

double Com_Filter_KalmanFilter(KalmanFilter_Struct *kf, double input);

#endif /* __COM_FILTER_H__ */
```

6.3.2 Com_Filter.c

```
#include "Com_Filter.h"

#define ALPHA 0.15 /* 一阶低通滤波 指数加权系数 */

/**
 * @description: 一阶低通滤波
 * 是一种常用的滤波器，用于去除高频噪声或高频成分，保留信号中的低频成分。
 * 在单片机应用中，一种简单且常见的低通滤波器是一阶无限脉冲响应（IIR）低通滤波器，
 * 通常实现为指数加权移动平均滤波器。
 * @param {int16_t} newValue 需要滤波的值
 * @param {int16_t} preFilteredValue 上一次滤波过的值
 * @return {*}
 */
int16_t Com_Filter_LowPass(int16_t newValue, int16_t preFilteredValue)
{
    return ALPHA * newValue + (1 - ALPHA) * preFilteredValue;
}

/* 卡尔曼滤波 https://www.mwrf.net/tech/basic/2023/30081.html */

/* 卡尔曼滤波参数 */
KalmanFilter_Struct kfs[3] = {
    {0.02, 0, 0, 0, 0.001, 0.543},
    {0.02, 0, 0, 0, 0.001, 0.543},
```

```
{0.02, 0, 0, 0, 0.001, 0.543}}};  
  
double Com_Filter_KalmanFilter(KalmanFilter_Struct *kf, double input)  
{  
    kf->Now_P = kf->LastP + kf->Q;  
    kf->Kg      = kf->Now_P / (kf->Now_P + kf->R);  
    kf->out     = kf->out + kf->Kg * (input - kf->out);  
    kf->LastP = (1 - kf->Kg) * kf->Now_P;  
    return kf->out;  
}
```

6.4 PID 模块

6.4.1 Com_PID.h

```
#ifndef __COM_PID_H__  
#define __COM_PID_H__  
  
typedef struct  
{  
    float kp;          // 比例增益 恢复是否及时 过大会震荡 过小恢复慢  
    float ki;          // 积分增益 消除稳态误差  
    float kd;          // 微分增益 消除过调  
    float desire;      // 目标值  
    float measure;     // 测量值  
    float integral;    // 误差积分  
    float last_error;  // 上一次的误差  
    float output;  
} PID_Struct;  
  
void Com_PID_Calculate(PID_Struct *pid, float dt);  
  
void Com_PID_Cascade(PID_Struct *outer, PID_Struct *inner, float dt);  
  
#endif /* __COM_PID_H__ */
```

6.4.2 Com_PID.c

```
#include "Com_PID.h"

void Com_PID_Calculate(PID_Struct *pid, float dt)
{
    // 1.计算当前误差
    float error = pid->measure - pid->desire;

    // 2.计算误差积分
    pid->integral = pid->integral + error * dt;

    // 3.计算误差变化率
    float derivative = (error - pid->last_error) / dt;
    pid->last_error = error;

    pid->output = pid->kp * error + pid->ki * pid->integral + pid->kd * derivative;
}

void Com_PID_Cascade(PID_Struct *outer, PID_Struct *inner, float dt)
{
    Com_PID_Calculate(outer, dt);
    inner->desire = outer->output;
    Com_PID_Calculate(inner, dt);
}
```

6.5 IMU 计算模块

四元素和欧拉角的解算

6.5.1 Com_IMU.h

```
#ifndef __COM_IMU_H__
```

```
#define __COM_IMU_H__

#include "Com_Types.h"
#include "math.h"

/* 表示四元数的结构体 */
typedef struct
{
    float q0;
    float q1;
    float q2;
    float q3;
} Quaternion_Struct;

extern float RtA;
extern float Gyro_G;
extern float Gyro_Gr;

void Com_IMU_GetEulerAngle(GyroAccel_Struct *gyroAccel,
                           EulerAngle_Struct *eulerAngle,
                           float dt);

float Com_IMU_GetNormAccZ(void);

#endif /* __COM_IMU_H__ */
```

6.5.2 Com_IMU.c

```
#include "Com_IMU.h"

/* =====欧拉角计算===== */
/* =====开始===== */

/* 计算欧拉角用到的 3 个参数 */
float RtA = 57.2957795f; // 弧度->度
// 陀螺仪初始化量程+-2000 度/秒于 1/(65536 / 4000) = 0.03051756*2
// float Gyro_G = 0.03051756f * 2;
```

```
float Gyro_G = 4000.0 / 65536; // 度/s

// 度每秒,转换弧度每秒则 2*0.03051756 * 0.0174533f = 0.0005326*2

// float Gyro_Gr = 0.0005326f * 2;

float Gyro_Gr = 4000.0 / 65536 / 180 * 3.1415926; // 弧度/s

#define sqa(Sq) (((float)Sq) * ((float)Sq)) /* 计算平方 */

/**
 * @description: 快速计算 1/sqrt(num)
 * @param {float} number
 */
static float Q_rsqrt(float number)
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y = number;
    i = *(long *)&y;
    i = 0x5f3759df - (i >> 1);
    y = *(float *)&i;
    y = y * (threehalfs - (x2 * y * y)); // 1st iteration (第一次牛顿迭代)
    return y;
}

static double normAccz; /* z 轴上的加速度 */

/**
 * @description: 根据 mpu 的 6 轴数据, 获取表征姿态的欧拉角
 * @param {GyroAccel_Struct} *gyroAccel mpu 的 6 轴数据
 * @param {EulerAngle_Struct} *EulerAngle 计算后得到的欧拉角
 * @param {float} dt 采样周期 (单位 s)
 * @return {*}
 */
void Com_IMU_GetEulerAngle(GyroAccel_Struct *gyroAccel,
                           EulerAngle_Struct *eulerAngle,
```

```
float dt)
{
    volatile struct V
    {
        float x;
        float y;
        float z;
    } Gravity, Acc, Gyro, AccGravity;

    static struct V GyroIntegError = {0};
    static float KpDef = 0.8f;
    static float KiDef = 0.0003f;
    static Quaternion_Struct NumQ = {1, 0, 0, 0};
    float q0_t, q1_t, q2_t, q3_t;
    // float NormAcc;
    float NormQuat;
    float HalfTime = dt * 0.5f;

    // 提取等效旋转矩阵中的重力分量
    Gravity.x = 2 * (NumQ.q1 * NumQ.q3 - NumQ.q0 * NumQ.q2);
    Gravity.y = 2 * (NumQ.q0 * NumQ.q1 + NumQ.q2 * NumQ.q3);
    Gravity.z = 1 - 2 * (NumQ.q1 * NumQ.q1 + NumQ.q2 * NumQ.q2);
    // 加速度归一化
    NormQuat = Q_rsqrt(squa(gyroAccel->accel.x) +
                      squa(gyroAccel->accel.y) +
                      squa(gyroAccel->accel.z));

    Acc.x = gyroAccel->accel.x * NormQuat;
    Acc.y = gyroAccel->accel.y * NormQuat;
    Acc.z = gyroAccel->accel.z * NormQuat;
    // 向量差乘得出的值
    AccGravity.x = (Acc.y * Gravity.z - Acc.z * Gravity.y);
    AccGravity.y = (Acc.z * Gravity.x - Acc.x * Gravity.z);
    AccGravity.z = (Acc.x * Gravity.y - Acc.y * Gravity.x);
```

```
// 再做加速度积分补偿角速度的补偿值
GyroIntegError.x += AccGravity.x * KiDef;
GyroIntegError.y += AccGravity.y * KiDef;
GyroIntegError.z += AccGravity.z * KiDef;

// 角速度融合加速度积分补偿值
Gyro.x = gyroAccel->gyro.x * Gyro_Gr + KpDef * AccGravity.x +
GyroIntegError.x; // 弧度制

Gyro.y = gyroAccel->gyro.y * Gyro_Gr + KpDef * AccGravity.y + GyroIntegError.y;
Gyro.z = gyroAccel->gyro.z * Gyro_Gr + KpDef * AccGravity.z + GyroIntegError.z;


// 一阶龙格库塔法, 更新四元数
q0_t = (-NumQ.q1 * Gyro.x - NumQ.q2 * Gyro.y - NumQ.q3 * Gyro.z) * HalfTime;
q1_t = (NumQ.q0 * Gyro.x - NumQ.q3 * Gyro.y + NumQ.q2 * Gyro.z) * HalfTime;
q2_t = (NumQ.q3 * Gyro.x + NumQ.q0 * Gyro.y - NumQ.q1 * Gyro.z) * HalfTime;
q3_t = (-NumQ.q2 * Gyro.x + NumQ.q1 * Gyro.y + NumQ.q0 * Gyro.z) * HalfTime;

NumQ.q0 += q0_t;
NumQ.q1 += q1_t;
NumQ.q2 += q2_t;
NumQ.q3 += q3_t;


// 四元数归一化
NormQuat = Q_rsqrt(squa(NumQ.q0) + squa(NumQ.q1) + squa(NumQ.q2) +
squa(NumQ.q3));

NumQ.q0 *= NormQuat;
NumQ.q1 *= NormQuat;
NumQ.q2 *= NormQuat;
NumQ.q3 *= NormQuat;


/*机体坐标系下的 Z 方向向量*/
float vecxZ = 2 * NumQ.q0 * NumQ.q2 - 2 * NumQ.q1 * NumQ.q3; /*矩阵(3,1)项*/
float vecyZ = 2 * NumQ.q2 * NumQ.q3 + 2 * NumQ.q0 * NumQ.q1; /*矩阵(3,2)项*/
float veczZ = 1 - 2 * NumQ.q1 * NumQ.q1 - 2 * NumQ.q2 * NumQ.q2; /*矩阵(3,3)项*/
```



```

float yaw_G = gyroAccel->gyro.z * Gyro_G;    // 将 Z 轴角速度陀螺仪值 转换为 Z 角度/秒
        Gyro_G 陀螺仪初始化量程+-2000 度每秒于 1 / (65536 / 4000) = 0.03051756*2
    if((yaw_G > 0.5f) || (yaw_G < -0.5))        // 数据太小可以认为是干扰，不是偏航
动作
    {
        eulerAngle->yaw += yaw_G * dt;    // 角速度积分成偏航角
    }

    eulerAngle->pitch = asin(vecxZ) * RtA;    // 俯仰角

    eulerAngle->roll = atan2f(vecyZ, veczZ) * RtA;    // 横滚角

    normAccz = gyroAccel->accel.x * vecxZ + gyroAccel->accel.y * vecyZ +
gyroAccel->accel.z * veczZ; /*Z 轴垂直方向上的加速度，此值涵盖了倾斜时在 Z 轴角速度的向量
和，不是单纯重力感应得出的值*/
}

/**
 * @description: 获取 Z 轴上的加速度 (如果已经倾斜,会考虑 z 轴上加速度的合成)
 * @return {*}
 */
float Com_IMU_GetNormAccZ(void)
{
    return normAccz;
}

/* =====欧拉角计算===== */
/* =====结束===== */

```

第 7 章 电源管理模块

7.1 STM32CubeMX 配置

Pin Name	Signal on Pin	GPIO outp...	GPIO mode	GPIO Pull...	Maximum ...	User Label	Modified
PA9	n/a	n/a	Input mode	Pull-up	n/a	SI24R1_IRQ	☒
PA11	n/a	Low	Output Pu...	No pull-up ...	Low	LED_RIGH...	☒
PA12	n/a	Low	Output Pu...	No pull-up ...	Low	LED_LEFT...	☒
PB0	n/a	High	Output Pu...	No pull-up ...	Low	PMW3901...	☒
PB2	n/a	High	Output Op...	No pull-up ...	Low	POWER_...	☒
PB3	n/a	n/a	Input mode	Pull-down	n/a	MPU6050_...	☒
PB12	n/a	High	Output Pu...	No pull-up ...	Low	VL53L4CD...	☒
PB13	n/a	n/a	Input mode	No pull-up ...	n/a	VL53L4CD...	☒
PB14	n/a	Low	Output Pu...	No pull-up ...	Low	LED_LEFT...	☒
PB15	n/a	Low	Output Pu...	No pull-up ...	Low	LED_RIGH...	☒

PB2 Configuration :

GPIO output level

High

GPIO mode

Output Open Drain

GPIO Pull-up/Pull-down

No pull-up and no pull-down

Maximum output speed

Low

User Label

POWER_KEY

7.2 硬件接口层

7.2.1 Inf_IP5305T.h

```

#ifndef __INF_IP5305T_H
#define __INF_IP5305T_H

#include "main.h"

#include "Common_Debug.h"
#include "Common_Delay.h"

void Inf_IP5305T_Open(void);

void Inf_IP5305T_Close(void);

#endif

```

7.2.2 Inf_IP5305T.c

```

#include "Inf_IP5305T.h"

/**
 * @description: 打开 IP5305T 芯片
 * IP5305T 可识别长按键和短按键操作，不需要按键时 PIN5 脚悬空。
 */

```

```
* key 对应的引脚为 PB2,应该设置为通用开漏模式:当设置为高电平时相当于引脚悬空 设置为低电平时相当于按下按键

* 短按唤醒时间为至少 80ms, 这个地方手册有误. 我们设置为 100ms

* @return {*}

*/

void Inf_IP5305T_Open(void)
{
    /* 短按 */
    HAL_GPIO_WritePin(POWER_KEY_GPIO_Port, POWER_KEY_Pin, GPIO_PIN_RESET);
    common_delay_ms(100);
    HAL_GPIO_WritePin(POWER_KEY_GPIO_Port, POWER_KEY_Pin, GPIO_PIN_SET);
}

/**
 * @description: 关闭 IP5305T 芯片
 * 在 1s 内连续两次短按
 * @return {*}
 */

void Inf_IP5305T_Close(void)
{
    /* 1 次短按 */
    HAL_GPIO_WritePin(POWER_KEY_GPIO_Port, POWER_KEY_Pin, GPIO_PIN_RESET);
    common_delay_ms(100);
    HAL_GPIO_WritePin(POWER_KEY_GPIO_Port, POWER_KEY_Pin, GPIO_PIN_SET);

    common_delay_ms(500);

    /* 2 次短按 */
    HAL_GPIO_WritePin(POWER_KEY_GPIO_Port, POWER_KEY_Pin, GPIO_PIN_RESET);
    common_delay_ms(100);
    HAL_GPIO_WritePin(POWER_KEY_GPIO_Port, POWER_KEY_Pin, GPIO_PIN_SET);
}
```

7.3 任务调度

```
/* 1. 电源控制任务 */  
  
void powerControlTask(void *arg);  
  
TaskHandle_t powerControlHandle;  
  
#define POWER_CONTORL_PRIORITY 9  
  
/*  
    1. 电源控制任务函数  
        需要每隔 20s 执行一次唤醒动作  
*/  
  
void powerControlTask(void *arg)  
{  
    debug_printfln("电源任务：开始调度...");  
    vTaskDelay(2000);  
    while(1)  
    {  
        /*  
            等待关机信号(比如遥控器传来的),  
            如果在 20s 没有获取到关机命令则重新唤醒电源  
        */  
        if(ulTaskNotifyTake(pdTRUE, 20 * 1000))  
        {  
            debug_printfln("电源:关机");  
            App_PowerControl_Close();  
        }  
        else  
        {  
            debug_printfln("电源:唤醒");  
            App_PowerControl_Open();  
        }  
    }  
}
```

第 8 章 灯控模块

8.1 STM32CubeMX 配置

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

Pin Na...	Si...	GPI...	GPIO mode	GPIO Pull-up/P...	Maxi...	User Label	Mo...
PB14	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED_LEFT_BOTTOM	☒
PA12	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED_LEFT_TOP	☒
PB15	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED_RIGHT_BOTTOM	☒
PA11	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED_RIGHT_TOP	☒

8.2 硬件接口层

8.2.1 Inf_LED.h

```

#ifndef __INF_LED_H
#define __INF_LED_H

#include "Common_Debug.h"
#include "main.h"
#include "Common_Config.h"
#include "Common_Delay.h"

extern LED_Struct leftTopLed;
extern LED_Struct leftBottomLed;
extern LED_Struct rightTopLed;
extern LED_Struct rightBottomLed;

void Inf_LED_SetStatus(LED_Struct *led);

#endif

```

8.2.2 Inf_LED.c

```

#include "Inf_LED.h"
/* 定义 4 个用到的 Led */
LED_Struct leftTopLed    = {LEFT_TOP, 0};
LED_Struct leftBottomLed = {LEFT_BOTTOM, 0};
LED_Struct rightTopLed   = {RIGHT_TOP, 0};
LED_Struct rightBottomLed = {RIGHT_BOTTOM, 0};

/**
 * @description: 设置 led 为指定的状态
 * 只能设定 打开, 关闭, 翻转.

```

```
* led 灯的闪烁频率需要使用者在应用层根据需要自己来实现
*/
void Inf_LED_SetStatus(LED_Struct *led)
{
    if(led->blink <= 1) /* 0=长亮或 1=长灭 */
    {
        if(led->location == LEFT_TOP)
        {
            HAL_GPIO_WritePin(LED_LEFT_TOP_GPIO_Port,
                               LED_LEFT_TOP_Pin,
                               (GPIO_PinState)led->blink);
        }
        else if(led->location == LEFT_BOTTOM)
        {
            HAL_GPIO_WritePin(LED_LEFT_BOTTOM_GPIO_Port,
                               LED_LEFT_BOTTOM_Pin,
                               (GPIO_PinState)led->blink);
        }
        else if(led->location == RIGHT_TOP)
        {
            HAL_GPIO_WritePin(LED_RIGHT_TOP_GPIO_Port,
                               LED_RIGHT_TOP_Pin,
                               (GPIO_PinState)led->blink);
        }
        else if(led->location == RIGHT_BOTTOM)
        {
            HAL_GPIO_WritePin(LED_RIGHT_BOTTOM_GPIO_Port,
                               LED_RIGHT_BOTTOM_Pin,
                               (GPIO_PinState)led->blink);
        }
    }
    else /* 翻转 */
    {
        if(led->location == LEFT_TOP)
```

```
{
    HAL_GPIO_TogglePin(LED_LEFT_TOP_GPIO_Port,
                        LED_LEFT_TOP_Pin);
}
else if(led->location == LEFT_BOTTOM)
{
    HAL_GPIO_TogglePin(LED_LEFT_BOTTOM_GPIO_Port,
                        LED_LEFT_BOTTOM_Pin);
}
else if(led->location == RIGHT_TOP)
{
    HAL_GPIO_TogglePin(LED_RIGHT_TOP_GPIO_Port,
                        LED_RIGHT_TOP_Pin);
}
else if(led->location == RIGHT_BOTTOM)
{
    HAL_GPIO_TogglePin(LED_RIGHT_BOTTOM_GPIO_Port,
                        LED_RIGHT_BOTTOM_Pin);
}
}
```

8.3 任务调度

```
Com_Status isConnected = Com_ERROR;
Com_Status isUnlocked  = Com_ERROR;
/* 3. led 控制任务 */
void      ledControlTask(void *arg);
TaskHandle_t ledControlHandle;
#define LED_CONTROL_PRIORITY 1
#define LED_CONTORL_EXECUTE_CYCLE 80
/* 3. led 控制任务函数 */
void ledControlTask(void *arg)
{
    vTaskDelay(1000);
```

```
debug_printfln("灯控任务：开始调度");

TickType_t pxPreviousWakeTime = xTaskGetTickCount();
uint32_t cnt = 0;
while(1)
{
    if(isUnlocked == Com_OK) /* 如果已经解锁，则 200ms 闪烁 */
    {
        if(joyStick.isFixHeightPoint)
        {
            leftTopLed.blink =
                leftBottomLed.blink =
                    rightTopLed.blink =
                        rightBottomLed.blink = 2;
        }
        else
        {
            leftTopLed.blink =
                leftBottomLed.blink =
                    rightTopLed.blink =
                        rightBottomLed.blink = 5;
        }
    }
    else if(isConnected == Com_OK) /* 如果已经连接，则 1s 闪烁 */
    {
        leftTopLed.blink =
            leftBottomLed.blink =
                rightTopLed.blink =
                    rightBottomLed.blink = 10;
    }
    else /* 其他情况 2s 闪烁 */
    {
        leftTopLed.blink =
            leftBottomLed.blink =
```



```
        rightTopLed.blink =
            rightBottomLed.blink = 20;
    }

    cnt++;
    if(leftTopLed.blink <= 1 || cnt % leftTopLed.blink == 0)
    {
        Inf_LED_SetStatus(&leftTopLed);
    }

    if(leftBottomLed.blink <= 1 || cnt % leftBottomLed.blink == 0)
    {
        Inf_LED_SetStatus(&leftBottomLed);
    }

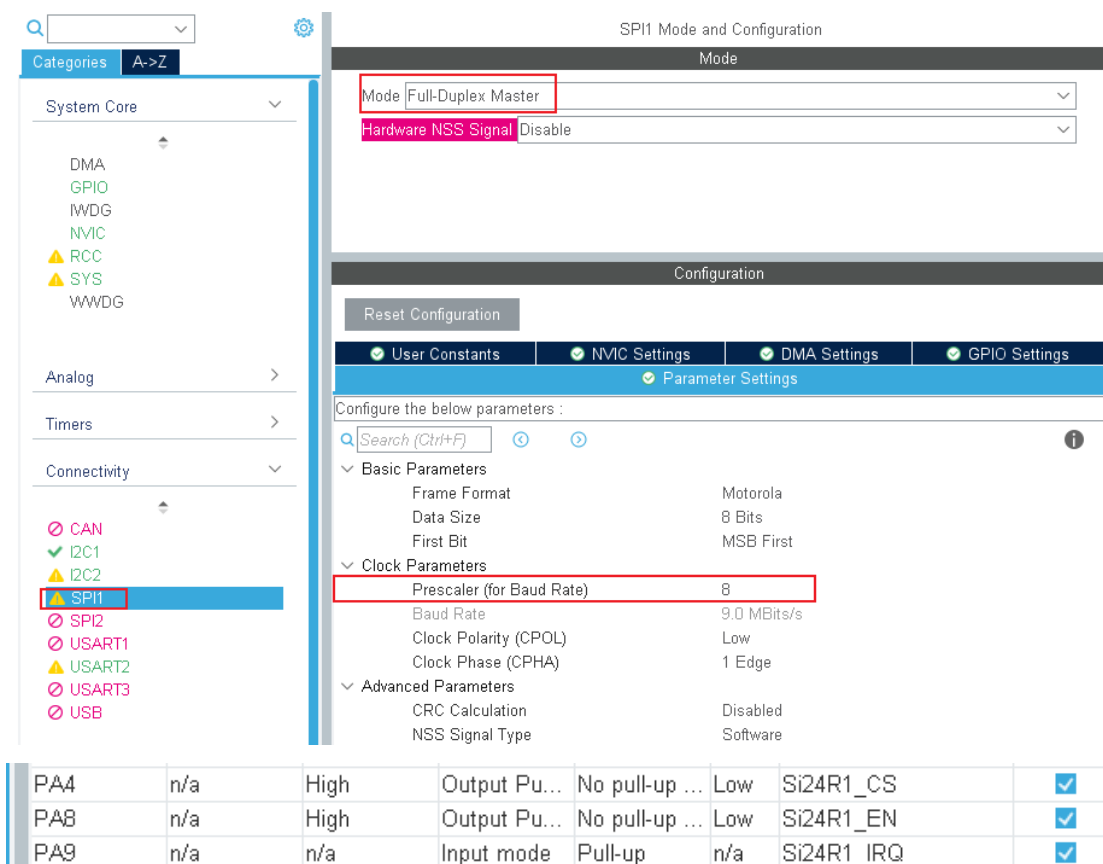
    if(rightTopLed.blink <= 1 || cnt % rightTopLed.blink == 0)
    {
        Inf_LED_SetStatus(&rightTopLed);
    }

    if(rightBottomLed.blink <= 1 || cnt % rightBottomLed.blink == 0)
    {
        Inf_LED_SetStatus(&rightBottomLed);
    }

    xTaskDelayUntil(&pxPreviousWakeTime, LED_CONTORL_EXECUTE_CYCLE);
}
}
```

第 9 章 通讯模块

9.1 STM32CubeMX 配置



PA4	n/a	High	Output Pu...	No pull-up ...	Low	Si24R1_CS	✓
PA8	n/a	High	Output Pu...	No pull-up ...	Low	Si24R1_EN	✓
PA9	n/a	n/a	Input mode	Pull-up	n/a	Si24R1_IRQ	✓

9.2 硬件接口层

9.2.1 Inf_Si24R1.h

```

#ifndef __nRF24L01P__
#define __nRF24L01P__

#include "spi.h"
#include "Com_Types.h"
#include "usart.h"

#define CHANNEL 40

#define SI24R1_CS_L HAL_GPIO_WritePin(SI24R1_CS_GPIO_Port, SI24R1_CS_Pin,
GPIO_PIN_RESET)

#define SI24R1_CS_H HAL_GPIO_WritePin(SI24R1_CS_GPIO_Port, SI24R1_CS_Pin,
GPIO_PIN_SET)

#define SI24R1_CE_L HAL_GPIO_WritePin(SI24R1_EN_GPIO_Port, SI24R1_EN_Pin,
GPIO_PIN_RESET)

#define SI24R1_CE_H HAL_GPIO_WritePin(SI24R1_EN_GPIO_Port, SI24R1_EN_Pin,
GPIO_PIN_SET)
    
```

```
#define TX_ADR_WIDTH    5    // 5 字节宽度的发送/接收地址

#define TX_PLOAD_WIDTH 17 // 数据通道有效数据宽度

//*****
*****//

// SPI(SI24R1) commands

#define SPI_READ_REG    0x00 // Define read command to register
#define SPI_WRITE_REG   0x20 // Define write command to register
#define RD_RX_PLOAD     0x61 // Define RX payload register address
#define WR_TX_PLOAD     0xA0 // Define TX payload register address
#define FLUSH_TX        0xE1 // Define flush TX register command
#define FLUSH_RX        0xE2 // Define flush RX register command
#define REUSE_TX_PL     0xE3 // Define reuse TX payload register command
#define NOP              0xFF // Define No Operation, might be used to read status
register

//*****
*****//

// SPI(SI24R1) registers(addresses)

#define CONFIG          0x00 // 'Config' register address
#define EN_AA           0x01 // 'Enable Auto Acknowledgment' register address
#define EN_RXADDR       0x02 // 'Enabled RX addresses' register address
#define SETUP_AW        0x03 // 'Setup address width' register address
#define SETUP_RETR      0x04 // 'Setup Auto. Retrans' register address
#define RF_CH           0x05 // 'RF channel' register address
#define RF_SETUP        0x06 // 'RF setup' register address
#define STATUS          0x07 // 'Status' register address
#define OBSERVE_TX      0x08 // 'Observe TX' register address
#define RSSI            0x09 // 'Received Signal Strength Indecator' register address
#define RX_ADDR_P0      0x0A // 'RX address pipe0' register address
#define RX_ADDR_P1      0x0B // 'RX address pipe1' register address
#define RX_ADDR_P2      0x0C // 'RX address pipe2' register address
#define RX_ADDR_P3      0x0D // 'RX address pipe3' register address
```

```
#define RX_ADDR_P4 0x0E // 'RX address pipe4' register address
#define RX_ADDR_P5 0x0F // 'RX address pipe5' register address
#define TX_ADDR    0x10 // 'TX address' register address
#define RX_PW_P0   0x11 // 'RX payload width, pipe0' register address
#define RX_PW_P1   0x12 // 'RX payload width, pipe1' register address
#define RX_PW_P2   0x13 // 'RX payload width, pipe2' register address
#define RX_PW_P3   0x14 // 'RX payload width, pipe3' register address
#define RX_PW_P4   0x15 // 'RX payload width, pipe4' register address
#define RX_PW_P5   0x16 // 'RX payload width, pipe5' register address
#define FIFO_STATUS 0x17 // 'FIFO Status Register' register address

//*****
//*****//

// STATUS Register
#define RX_DR 0x40 /**/
#define TX_DS 0x20
#define MAX_RT 0x10

//*****
//*****//

//                               FUNCTION's
PROTOTYPES                               //

//*****
//*****//

// SI24R1 API Functions
uint8_t Int_SI24R1_WriteByte(uint8_t cmd, uint8_t value);
uint8_t Int_SI24R1_WriteBytes(uint8_t cmd, const uint8_t *pBuf, uint8_t bytes);
uint8_t Int_SI24R1_ReadByte(uint8_t cmd);
uint8_t Int_SI24R1_ReadBytes(uint8_t cmd, uint8_t *pBuf, uint8_t bytes);

void Int_SI24R1_RXMode(void);
void Int_SI24R1_TXMode(void);
uint8_t Int_SI24R1_RxPacket(uint8_t *rxbuf);
uint8_t Int_SI24R1_TxPacket(uint8_t *txbuf);
```

```
void Int_SI24R1_Init(void);

//*****

*****//

#endif
```

9.2.2 Inf_Si24R1.c

```
#include "Inf_Si24R1.h"
#include "FreeRTOS.h"
#include "task.h"

uint8_t TX_ADDRESS[TX_ADR_WIDTH] = {0x0A, 0x01, 0x07, 0x0E, 0x01}; // 定义一个静态发送地址

static uint8_t SPI_RW(uint8_t byte)
{
    uint8_t result;
    taskENTER_CRITICAL();
    HAL_SPI_TransmitReceive(&hspi1, &byte, &result, 1, 1000);
    taskEXIT_CRITICAL();
    return result;
}

/*****
函数功能：写寄存器的值（单字节）
入口参数：reg:寄存器映射地址（格式：SPI_WRITE_REG | reg）
           value:寄存器的值
返回 值：状态寄存器的值
*****/

uint8_t Int_SI24R1_WriteByte(uint8_t reg, uint8_t value)
{
    uint8_t status;
```

```
    SI24R1_CS_L;

    status = SPI_RW(reg);

    SPI_RW(value);

    SI24R1_CS_H;

    return (status);
}
```

/******

函数功能：写寄存器的值（多字节）

入口参数：**reg**:寄存器映射地址（格式：SPI_WRITE_REG | reg）

pBuf:写数据首地址

bytes:写数据字节数

返回 值：状态寄存器的值

*****/

```
uint8_t Int_SI24R1_WriteBytes(uint8_t reg, const uint8_t *pBuf, uint8_t bytes)
```

```
{
    uint8_t status, byte_ctr;

    SI24R1_CS_L;

    status = SPI_RW(reg);

    for (byte_ctr = 0; byte_ctr < bytes; byte_ctr++)

        SPI_RW(*pBuf++);

    SI24R1_CS_H;

    return (status);
}
```

/******

函数功能：读取寄存器的值（单字节）

入口参数：**reg**:寄存器映射地址（格式：SPI_READ_REG | reg）

返回 值：寄存器值

*****/

```
uint8_t Int_SI24R1_ReadByte(uint8_t reg)
```

```
{
    uint8_t value;

    SI24R1_CS_L;
    SPI_RW(reg);
    value = SPI_RW(0);
    SI24R1_CS_H;

    return (value);
}
```

函数功能：读取寄存器的值（多字节）

入口参数：reg:寄存器映射地址（SPI_READ_REG | reg）

 pBuf:接收缓冲区的首地址

 bytes:读取字节数

返回 值：状态寄存器的值

*****/

```
uint8_t Int_SI24R1_ReadBytes(uint8_t reg, uint8_t *pBuf, uint8_t bytes)
{
    uint8_t status, byte_ctr;

    SI24R1_CS_L;
    status = SPI_RW(reg);
    for (byte_ctr = 0; byte_ctr < bytes; byte_ctr++)
        pBuf[byte_ctr] = SPI_RW(0); // 读取数据，低字节在前
    SI24R1_CS_H;

    return (status);
}
```

函数功能：SI24R1 接收模式初始化

入口参数：无

返回 值：无

```

*****/

void Int_SI24R1_RXMode(void)
{
    SI24R1_CE_L;

    Int_SI24R1_WriteBytes(SPI_WRITE_REG + RX_ADDR_P0, TX_ADDRESS, TX_ADR_WIDTH); //
接收设备接收通道 0 使用和发送设备相同的发送地址

    Int_SI24R1_WriteByte(SPI_WRITE_REG + EN_AA, 0x01); //
使能接收通道 0 自动应答

    Int_SI24R1_WriteByte(SPI_WRITE_REG + EN_RXADDR, 0x01); //
使能接收通道 0

    Int_SI24R1_WriteByte(SPI_WRITE_REG + RF_CH, CHANNEL); //
选择射频通道 0x40

    Int_SI24R1_WriteByte(SPI_WRITE_REG + RX_PW_P0, TX_PLOAD_WIDTH); //
接收通道 0 选择和发送通道相同有效数据宽度

    Int_SI24R1_WriteByte(SPI_WRITE_REG + RF_SETUP, 0x0f); //
数据传输率 2Mbps, 发射功率 7dBm

    Int_SI24R1_WriteByte(SPI_WRITE_REG + CONFIG, 0x0f); //
CRC 使能, 16 位 CRC 校验, 上电, 接收模式

    Int_SI24R1_WriteByte(SPI_WRITE_REG + STATUS, 0xff); //
清除所有的中断标志位

    SI24R1_CE_H; //
拉高 CE 启动接收设备
}

/*****

函数功能：SI24R1 发送模式初始化
入口参数：无
返回 值：无

*****/

void Int_SI24R1_TXMode(void)
{
    SI24R1_CE_L;
```



```

    Int_SI24R1_WriteBytes(SPI_WRITE_REG + TX_ADDR, TX_ADDRESS, TX_ADR_WIDTH);    //
写入发送地址

    Int_SI24R1_WriteBytes(SPI_WRITE_REG + RX_ADDR_P0, TX_ADDRESS, TX_ADR_WIDTH); //
为了应答接收设备，接收通道 0 地址和发送地址相同

    Int_SI24R1_WriteByte(SPI_WRITE_REG + EN_AA, 0x01);        // 使能接收通道 0 自动应答
    Int_SI24R1_WriteByte(SPI_WRITE_REG + EN_RXADDR, 0x01);    // 使能接收通道 0
    Int_SI24R1_WriteByte(SPI_WRITE_REG + SETUP_RETR, 0x0a);    // 自动重发延时等待
250us+86us，自动重发 10 次

    Int_SI24R1_WriteByte(SPI_WRITE_REG + RF_CH, CHANNEL);      // 选择射频通道 0x40
    Int_SI24R1_WriteByte(SPI_WRITE_REG + RF_SETUP, 0x0f);      // 数据传输率 2Mbps，发射
功率 7dBm

    Int_SI24R1_WriteByte(SPI_WRITE_REG + CONFIG, 0x0e);        // CRC 使能，16 位 CRC 校
验，上电

    SI24R1_CE_H;                                                // SI24R1_CE_H;
}

/*****
函数功能：读取接收数据
入口参数：rxbuf:接收数据存放首地址
返回  值：0:接收到数据
        1:没有接收到数据
*****/
uint8_t Int_SI24R1_RxPacket(uint8_t *rxbuf)
{
    uint8_t state;

    state = Int_SI24R1_ReadByte(STATUS);                        // 读取状态寄存器的值
    Int_SI24R1_WriteByte(SPI_WRITE_REG + STATUS, state);        // 清除 RX_DS 中断标志

    if (state & RX_DR) // 接收到数据
    {
        Int_SI24R1_ReadBytes(RD_RX_PLOAD, rxbuf, TX_PLOAD_WIDTH); // 读取数据
        Int_SI24R1_WriteByte(FLUSH_RX, 0xff);                    // 清除 RX FIFO 寄
存器
    }
}

```

```

        return 0;
    }

    return 1; // 没收到任何数据
}

/*****
函数功能：发送一个数据包
入口参数：txbuf:要发送的数据
返回  值：
        0:发送成功
        1:发送失败
*****/

uint8_t Int_SI24R1_TxPacket(uint8_t *txbuf)
{
    uint8_t state = 0x00;

    SI24R1_CE_L; // CE 拉低，使能
SI24R1 配置

    Int_SI24R1_WriteBytes(WR_TX_PLOAD, txbuf, TX_PLOAD_WIDTH); // 写数据到 TX FIFO,32
个字节

    SI24R1_CE_H; // CE 置高，使能发送

    // 等待发送完成
    while ((state & (MAX_RT | TX_DS)) == 0x00)
    {
        // 读取状态寄存器的值
        state = Int_SI24R1_ReadByte(STATUS);
    }

    Int_SI24R1_WriteByte(SPI_WRITE_REG + STATUS, state); // 清除 TX_DS 或 MAX_RT 中断标
志

    if (state & MAX_RT) // 达到最大重发次数
    {
        Int_SI24R1_WriteByte(FLUSH_TX, 0xff); // 清除 TX FIFO 寄存器

        return 1;
    }
}

```

```
    if (state & TX_DS) // 发送完成
    {
        return 0;
    }
    return 1; // 发送失败
}

static Com_Status Int_SI24R1_SelfCheck()
{
    Int_SI24R1_WriteBytes(SPI_WRITE_REG + TX_ADDR, TX_ADDRESS, TX_ADR_WIDTH);
    uint8_t rx_buffer[TX_ADR_WIDTH];
    Int_SI24R1_ReadBytes(SPI_READ_REG + TX_ADDR, rx_buffer, TX_ADR_WIDTH);

    for (uint8_t i = 0; i < TX_ADR_WIDTH; i++)
    {
        if (rx_buffer[i] != TX_ADDRESS[i])
        {
            return COM_ERROR;
        }
    }
    return COM_OK;
}

void Int_SI24R1_Init(void)
{
    vTaskDelay(100);
    // 等待自检成功
    while (Int_SI24R1_SelfCheck() != COM_OK)
    {
        printf("SI24R1 self check failed!");
    }
    printf("SI24R1 self check success!\n");

    // 设置为接收模式
```

```
Int_SI24R1_RXMode();  
  
printf("RXMode success!\n");  
}
```

9.3 应用层

9.3.1 App_Communication.h

```
#ifndef __APP_COMMUNICATION_H  
#define __APP_COMMUNICATION_H  
  
#include "Common_Debug.h"  
#include "Inf_Si24R1.h"  
#include "Common_Config.h"  
  
void App_Communication_Start(void);  
Com_Status App_Communication_ReadRemoteData(void);  
Com_Status App_Communication_ConnectCheck(Com_Status isReadData);  
#endif
```

9.3.2 App_Communication.c

```
#include "App_Communication.h"  
  
JoyStick_Struct joyStick;  
  
/**  
 * @description: 启动通讯模块  
 * @return {*}  
 */  
  
void App_Communication_Start(void)  
{  
    debug_printfln("通讯模块开始启动");  
  
    /* 1. 等待自检通过 */  
    debug_printfln("    2.4G 模块开始自检");  
    HAL_Delay(1000);  
    while(Inf_Si24R1_Check() == 1);  
    debug_printfln("    2.4G 模块自检通过");  
}
```

```
/* 2. 把 2.4G 模块设置为接收模式 */
debug_printfln("    2.4G 模块设置为接收模式");
Inf_Si24R1_RXMode();
debug_printfln("通讯模块完成启动");
}

/**
 * @description: 接收遥控发来的数据, 把收到的数据, 并对收到的数据做校验
 * @return {*}
 */
Com_Status App_Communication_ReadRemoteData(void)
{
    if(Inf_Si24R1_RxPacket(RX_BUFF)) return Com_OTHER; /* 没有收到数据 */

    /* 1. 检测帧头 */
    uint8_t index = 0;
    if(RX_BUFF[index++] != FRAME_0 ||
        RX_BUFF[index++] != FRAME_1 ||
        RX_BUFF[index++] != FRAME_2) return Com_OTHER; /* 帧头与预定义的不一致 */

    /* 2. 检测校验和 */
    /* 2.1 获取载荷字节数 */
    uint8_t payLoad = RX_BUFF[index++];
    /* 2.2 计算校验和 */
    uint32_t calcChecksum = 0;
    for(uint8_t i = 0; i < index + payLoad; i++)
    {
        calcChecksum += RX_BUFF[i];
    }
    /* 2.3 获取校验和 */
    uint32_t recvChecksum = (RX_BUFF[index++ + payLoad] << 24) |
        (RX_BUFF[index++ + payLoad] << 16) |
        (RX_BUFF[index++ + payLoad] << 8) |
        (RX_BUFF[index++ + payLoad]);
    /* 2.4 判断获取的校验和与计算的校验和是否相等 */
}
```

```
    if(calcChecksum != recvChecksum) return Com_OTHER;

    return Com_OK;
}

Com_Status App_Communication_ConnectCheck(Com_Status isReadData)
{
    /* 通讯任务调度周期为 4ms，所以，每计数一次表示 4ms */
    static uint16_t noRecvDataCnt = 0; /* 用来记录收不到数据的次数 */
    if(isReadData == Com_OK)
    {
        noRecvDataCnt = 0; /* 一旦收到数据，则清零 i */
        /* 把收到的数据封装到摇杆结构体中 */
        joyStick.THR = (RX_BUFF[4] << 8 | RX_BUFF[5]);
        joyStick.YAW = (RX_BUFF[6] << 8 | RX_BUFF[7]);
        joyStick.ROL = (RX_BUFF[8] << 8 | RX_BUFF[9]);
        joyStick.PIT = (RX_BUFF[10] << 8 | RX_BUFF[11]);

        /* 收到定高命令，就对当前的定高状态取反 */
        if(RX_BUFF[12])
        {
            joyStick.isFixHeightPoint = !joyStick.isFixHeightPoint;
        }

        joyStick.isPowerDonw = RX_BUFF[13];

        return Com_OK; /* 如果收到数据,证明连接成功 */
    }

    noRecvDataCnt++; /* 对收不到数据的次数 +1 */
    if(noRecvDataCnt >= 250) /* 如果连续 250 次(1s)收不到数据，则表示失联 */
    {
        noRecvDataCnt = 250;
        return Com_ERROR;
    }
}
```

```

    }

    return Com_OTHER;
}

```

第 10 章 飞控模块

10.1 STM32CubeMX 配置

10.1.1 马达驱动配置

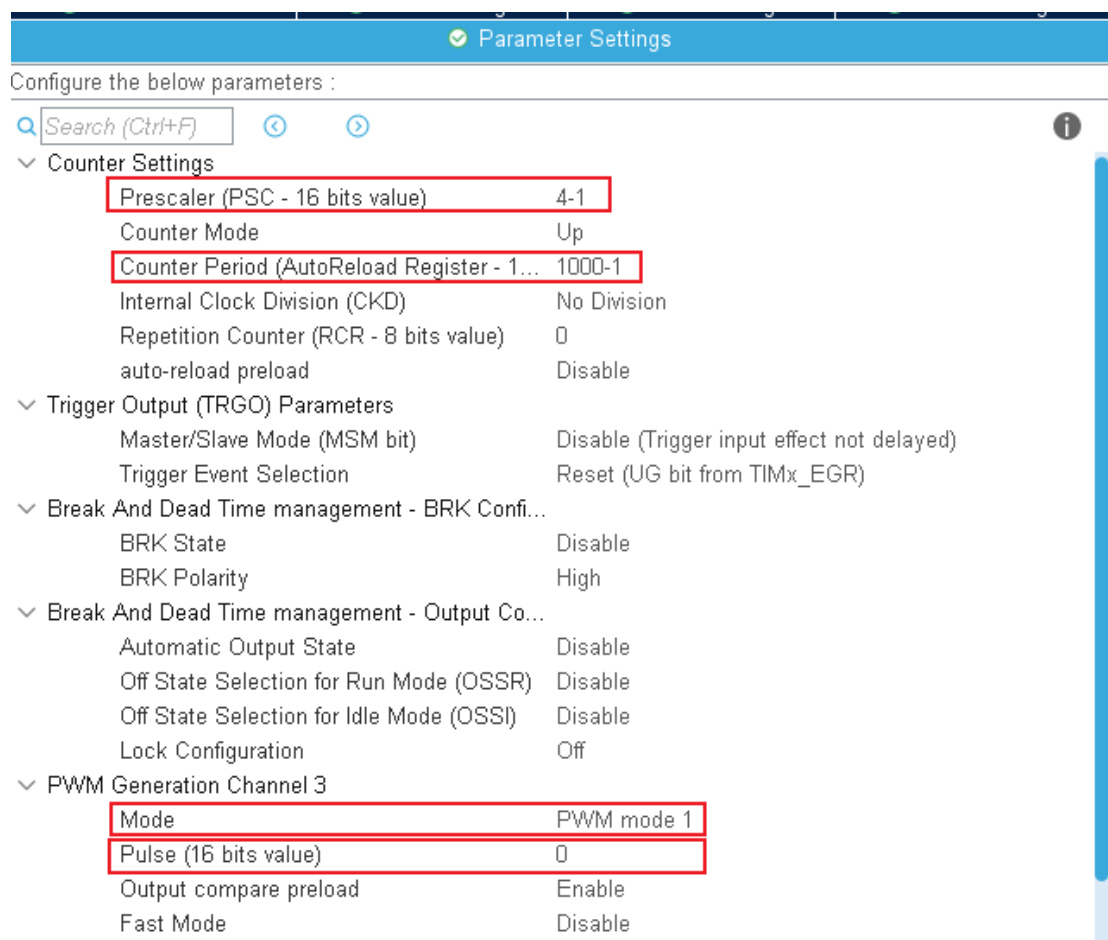
配置 4 个定时器

10.1.1.1 定时器 1 配置

TIM1 Mode and Configuration

Mode

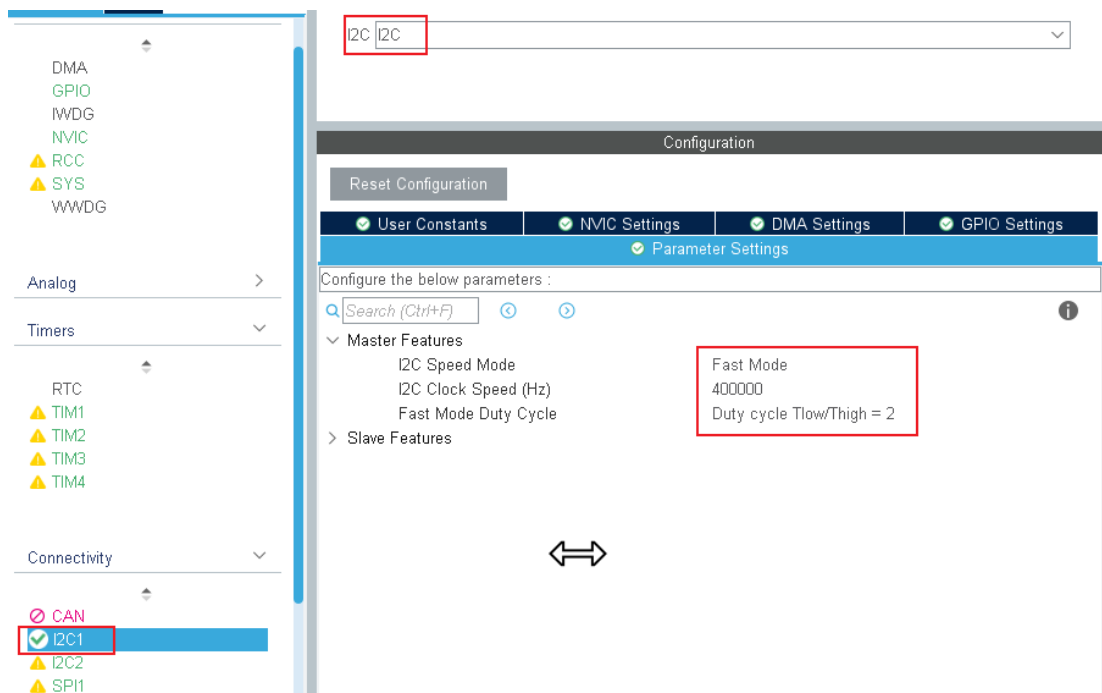
Slave Mode	Disable
Trigger Source	Disable
Clock Source	Internal Clock
Channel1	Disable
Channel2	Disable
Channel3	PWM Generation CH3
Channel4	Disable
Combined Channels	Disable
☐ Activate-Break-Input	
☐ Use ETR as Clearing Source	
☐ XOR activation	



10.1.1.2 其他配置

其他定时器与定时器配置类似, 请参考原理图逐一配置

10.1.2 MPU60 配置



10.2 硬件接口层

10.2.1 电机

10.2.1.1 Inf_Motor.h

```
#ifndef __INF_MOTOR_H
#define __INF_MOTOR_H

#include "Common_Debug.h"
#include "tim.h"
#include "Common_Config.h"

extern Motor_Struct leftTopMotor;
extern Motor_Struct leftBottomMotor;
extern Motor_Struct rightTopMotor;
extern Motor_Struct rightBottomMotor;

void Inf_Motor_Init(void);
void Inf_Motor_SetMotorSpeed(Motor_Struct *motor);
void Inf_Motor_AllWork(void);
#endif
```

10.2.1.2 Inf_Motor.c

```
#include "Inf_Motor.h"

Motor_Struct leftTopMotor    = {LEFT_TOP, 0};
Motor_Struct leftBottomMotor = {LEFT_BOTTOM, 0};
Motor_Struct rightTopMotor   = {RIGHT_TOP, 0};
Motor_Struct rightBotomMotor = {RIGHT_BOTTOM};

/**
 * @description: 4 个驱动马达的初始化
 * @return {*}
 */
void Inf_Motor_Init(void)
{
    debug_printfln("    马达：开始初始化");

    /* left-top: TIM3_CH1 */
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
    /* left-bottom: TIM4_CH4*/
    HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);
    /* right-top: TIM2_CH2 */
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
    /* right-bottom: TIM1_CH3 */
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);

    /* 4 个马达速度均初始化为 0 */
    debug_printfln("    马达：4 个均初始化为速度 0");

    leftTopMotor.speed =
        leftBottomMotor.speed =
            rightTopMotor.speed =
                rightBotomMotor.speed = 0;

    Inf_Motor_AllWork();
    debug_printfln("    马达：结束初始化");
}
```

```
/**
 * @description: 控制所有马达运转起来
 * @return {*}
 */
void Inf_Motor_AllWork(void)
{
    Inf_Motor_SetMotorSpeed(&leftTopMotor);
    Inf_Motor_SetMotorSpeed(&leftBottomMotor);
    Inf_Motor_SetMotorSpeed(&rightTopMotor);
    Inf_Motor_SetMotorSpeed(&rightBottomMotor);
}

/**
 * @description: 给指定的马达设置运转速度
 * @param {Motor_Struct} motor 要设置速度的马达
 * @return {*}
 */
void Inf_Motor_SetMotorSpeed(Motor_Struct *motor)
{
    motor->speed = motor->speed >= 1000 ? 1000 : motor->speed;

    if(motor->location == LEFT_TOP)
    {
        __HAL_TIM_SetCompare(&htim3, TIM_CHANNEL_1, motor->speed);
    }
    else if(motor->location == LEFT_BOTTOM)
    {
        __HAL_TIM_SetCompare(&htim4, TIM_CHANNEL_4, motor->speed);
    }
    else if(motor->location == RIGHT_TOP)
    {
        __HAL_TIM_SetCompare(&htim2, TIM_CHANNEL_2, motor->speed);
    }
    else if(motor->location == RIGHT_BOTTOM)
```

```
{  
    __HAL_TIM_SetCompare(&htim1, TIM_CHANNEL_3, motor->speed);  
}  
}
```

10.2.2 MPU6050

10.2.2.1 Inf_MPU6050.h

```
#ifndef __INF_MPU6050_H  
#define __INF_MPU6050_H  
  
#include "Common_Debug.h"  
#include "i2c.h"  
#include "Common_Config.h"  
  
#define MPU_IIC_ADDR_BASE 0x68 /* AD0=0=>0x68 AD0=1=>0x69 */  
#define MPU_IIC_ADDR_W (0x68 << 1)  
#define MPU_IIC_ADDR_R ((0x68 << 1) + 1)  
  
#define MPU_SELF_TESTX_REG 0X0D // 自检寄存器 X  
#define MPU_SELF_TESTY_REG 0X0E // 自检寄存器 Y  
#define MPU_SELF_TESTZ_REG 0X0F // 自检寄存器 Z  
#define MPU_SELF_TESTA_REG 0X10 // 自检寄存器 A  
#define MPU_SAMPLE_RATE_REG 0X19 // 采样频率分频器  
#define MPU_CFG_REG 0X1A // 配置寄存器  
#define MPU_GYRO_CFG_REG 0X1B // 陀螺仪配置寄存器  
#define MPU_ACCEL_CFG_REG 0X1C // 加速度计配置寄存器  
#define MPU_MOTION_DET_REG 0X1F // 运动检测阈值设置寄存器  
#define MPU_FIFO_EN_REG 0X23 // FIFO 使能寄存器  
#define MPU_I2CMST_CTRL_REG 0X24 // IIC 主机控制寄存器  
#define MPU_I2CSLV0_ADDR_REG 0X25 // IIC 从机 0 器件地址寄存器  
#define MPU_I2CSLV0_REG 0X26 // IIC 从机 0 数据地址寄存器  
#define MPU_I2CSLV0_CTRL_REG 0X27 // IIC 从机 0 控制寄存器  
#define MPU_I2CSLV1_ADDR_REG 0X28 // IIC 从机 1 器件地址寄存器  
#define MPU_I2CSLV1_REG 0X29 // IIC 从机 1 数据地址寄存器  
#define MPU_I2CSLV1_CTRL_REG 0X2A // IIC 从机 1 控制寄存器
```

```
#define MPU_I2CSLV2_ADDR_REG 0X2B // IIC 从机 2 器件地址寄存器
#define MPU_I2CSLV2_REG 0X2C // IIC 从机 2 数据地址寄存器
#define MPU_I2CSLV2_CTRL_REG 0X2D // IIC 从机 2 控制寄存器
#define MPU_I2CSLV3_ADDR_REG 0X2E // IIC 从机 3 器件地址寄存器
#define MPU_I2CSLV3_REG 0X2F // IIC 从机 3 数据地址寄存器
#define MPU_I2CSLV3_CTRL_REG 0X30 // IIC 从机 3 控制寄存器
#define MPU_I2CSLV4_ADDR_REG 0X31 // IIC 从机 4 器件地址寄存器
#define MPU_I2CSLV4_REG 0X32 // IIC 从机 4 数据地址寄存器
#define MPU_I2CSLV4_DO_REG 0X33 // IIC 从机 4 写数据寄存器
#define MPU_I2CSLV4_CTRL_REG 0X34 // IIC 从机 4 控制寄存器
#define MPU_I2CSLV4_DI_REG 0X35 // IIC 从机 4 读数据寄存器

#define MPU_I2CMST_STA_REG 0X36 // IIC 主机状态寄存器
#define MPU_INTBP_CFG_REG 0X37 // 中断/旁路设置寄存器
#define MPU_INT_EN_REG 0X38 // 中断使能寄存器
#define MPU_INT_STA_REG 0X3A // 中断状态寄存器

#define MPU_ACCEL_XOUTH_REG 0X3B // 加速度值,X 轴高 8 位寄存器
#define MPU_ACCEL_XOUTL_REG 0X3C // 加速度值,X 轴低 8 位寄存器
#define MPU_ACCEL_YOUTH_REG 0X3D // 加速度值,Y 轴高 8 位寄存器
#define MPU_ACCEL_YOUTL_REG 0X3E // 加速度值,Y 轴低 8 位寄存器
#define MPU_ACCEL_ZOUTH_REG 0X3F // 加速度值,Z 轴高 8 位寄存器
#define MPU_ACCEL_ZOUTL_REG 0X40 // 加速度值,Z 轴低 8 位寄存器

#define MPU_TEMP_OUTH_REG 0X41 // 温度值高八位寄存器
#define MPU_TEMP_OUTL_REG 0X42 // 温度值低 8 位寄存器

#define MPU_GYRO_XOUTH_REG 0X43 // 陀螺仪值,X 轴高 8 位寄存器
#define MPU_GYRO_XOUTL_REG 0X44 // 陀螺仪值,X 轴低 8 位寄存器
#define MPU_GYRO_YOUTH_REG 0X45 // 陀螺仪值,Y 轴高 8 位寄存器
#define MPU_GYRO_YOUTL_REG 0X46 // 陀螺仪值,Y 轴低 8 位寄存器
#define MPU_GYRO_ZOUTH_REG 0X47 // 陀螺仪值,Z 轴高 8 位寄存器
#define MPU_GYRO_ZOUTL_REG 0X48 // 陀螺仪值,Z 轴低 8 位寄存器
```

```
#define MPU_I2CSLV0_DO_REG 0X63 // IIC 从机 0 数据寄存器
#define MPU_I2CSLV1_DO_REG 0X64 // IIC 从机 1 数据寄存器
#define MPU_I2CSLV2_DO_REG 0X65 // IIC 从机 2 数据寄存器
#define MPU_I2CSLV3_DO_REG 0X66 // IIC 从机 3 数据寄存器

#define MPU_I2CMST_DELAY_REG 0X67 // IIC 主机延时管理寄存器
#define MPU_SIGPATH_RST_REG 0X68 // 信号通道复位寄存器
#define MPU_MDETECT_CTRL_REG 0X69 // 运动检测控制寄存器
#define MPU_USER_CTRL_REG 0X6A // 用户控制寄存器
#define MPU_PWR_MGMT1_REG 0X6B // 电源管理寄存器 1
#define MPU_PWR_MGMT2_REG 0X6C // 电源管理寄存器 2
#define MPU_FIFO_CNTH_REG 0X72 // FIFO 计数寄存器高八位
#define MPU_FIFO_CNTL_REG 0X73 // FIFO 计数寄存器低八位
#define MPU_FIFO_RW_REG 0X74 // FIFO 读写寄存器
#define MPU_DEVICE_ID_REG 0X75 // 器件 ID 寄存器

extern GyroAccel_Struct gyroAccel;

uint8_t Inf_MPU6050_ReadId(void);

void Inf_MPU6050_Init(void);
void Inf_MPU6050_GetGyro(Gyro_Struct *gyro);
void Inf_MPU6050_GetAccel(Accel_Struct *accel);
void Inf_MPU6050_GetGyroAccel(GyroAccel_Struct *gyroAccel);
#endif
```

10.2.2.2 Inf_MPU6050.c

```
#include "Inf_MPU6050.h"

GyroAccel_Struct gyroAccel;

#define I2C_DELAY 2000

#define abs(x) ((x) >= 0 ? (x) : (-x))

static void Inf_MPU6050_WriteReg(uint8_t reg, uint8_t value);
static uint8_t Inf_MPU6050_ReadReg(uint8_t reg);
```

```
static void      Inf_MPU6050_Calibrate(void);

/* 存储 6 个偏移值 */
static GyroAccel_Struct mpuOffset;
static uint8_t      isCalibrated = 0; /* 表示是否已经校准完成 */
/**
 * @description: 读取设备 id, 用于检测驱动是否正常
 * @return {*} 读取到的 id 值. 根据寄存器描述, 读取到的其实就是它的 7 位 I2C 地址 0x68
 */
uint8_t Inf_MPU6050_ReadId(void)
{
    return Inf_MPU6050_ReadReg(MPU_DEVICE_ID_REG);
}
/**
 * @description: 初始化 MPU6050 芯片
 * @return {*}
 */
void Inf_MPU6050_Init(void)
{
    debug_printfln("    陀螺仪: 开始初始化");

    /* 1. 复位 =>休眠=>唤醒*/
    Inf_MPU6050_WriteReg(MPU_PWR_MGMT1_REG, 0x80);
    HAL_Delay(200); /* 延时 100ms 再唤醒 */
    /*唤醒: 复位后自动进入睡眠模式, 所以需要先唤醒 */
    Inf_MPU6050_WriteReg(MPU_PWR_MGMT1_REG, 0x00);

    /* 2. 设置量程 */
    /* 2.1 设置角速度量程 +-2000°/s*/
    Inf_MPU6050_WriteReg(MPU_GYRO_CFG_REG, 3 << 3);
    /* 2.2 设置加速度量程 -2g*/
    Inf_MPU6050_WriteReg(MPU_ACCEL_CFG_REG, 0 << 3);

    /* 3. 其他配置: 关闭 FIFO 中断 第二 I2C */
}
```

```
Inf_MPU6050_WriteReg(MPU_USER_CTRL_REG, 0x00);

Inf_MPU6050_WriteReg(MPU_FIFO_EN_REG, 0x00);

Inf_MPU6050_WriteReg(MPU_INT_EN_REG, 0x00);

/* 4. 设置设备的系统时钟源 PLL 参考陀螺仪 x 轴*/
Inf_MPU6050_WriteReg(MPU_PWR_MGMT1_REG, 0x01);

/* 5. 设置低通滤波器和采样率 */
/* 5.1 设置采样率 500Hz = 1000/(1 + div) => div = 1 */
Inf_MPU6050_WriteReg(MPU_SAMPLE_RATE_REG, 1);
/* 5.2 设置低通滤波器 香农采样定律: 采样率>=2*最高频率 最高频率<=采样率/2 最高频率<=250 */
Inf_MPU6050_WriteReg(MPU_CFG_REG, 1 << 0);

/* 6. 启动加速度传感器和陀螺仪 */
Inf_MPU6050_WriteReg(MPU_PWR_MGMT2_REG, 0x00);

/* 7. 对陀螺仪做校准 */
isCalibrated = 0;
Inf_MPU6050_Calibrate();

debug_printfln("    陀螺仪: 完成初始化");
}

/**
 * @description: 向 mpu 指定的寄存器写入一个字节
 * @param {uint8_t} reg 要写入的寄存器地址
 * @param {uint8_t} value 要写入的具体的值
 * @return {*}
 */
static void Inf_MPU6050_WriteReg(uint8_t reg, uint8_t value)
{
    HAL_I2C_Mem_Write(&hi2c1,
                      MPU_IIC_ADDR_W,
```



```
        reg,
        I2C_MEMADD_SIZE_8BIT,
        &value,
        1,
        I2C_DELAY);
}

/**
 * @description: 从指定的寄存器地址读取一个值
 * @param {uint8_t} reg 要读取的寄存器地址
 * @return {*} 读取到的值
 */
static uint8_t Inf_MPU6050_ReadReg(uint8_t reg)
{
    uint8_t result = 0x00;
    HAL_I2C_Mem_Read(&hi2c1,
                     MPU_IIC_ADDR_R,
                     reg,
                     I2C_MEMADD_SIZE_8BIT,
                     &result,
                     1,
                     I2C_DELAY);

    return result;
}

/**
 * @description: 从指定的寄存器开始连续读取多个寄存器的值
 * @param {uint8_t} reg 开始的寄存器地址
 * @param {uint8_t} len 读取的寄存器的个数
 * @param {uint8_t} data 存储读取到值
 * @return {*}
 */
static void Inf_MPU6050_ReadRegs(uint8_t reg, uint8_t len, uint8_t data[])
```

```
{
    HAL_I2C_Mem_Read(&hi2c1,
                     MPU_IIC_ADDR_R,
                     reg,
                     I2C_MEMADD_SIZE_8BIT,
                     data,
                     len,
                     2000);
}

/**
 * @description: 获取陀螺仪的各轴的角速度
 * @param {Gyro_Struct} *gyro
 * @return {*}
 */
void Inf_MPU6050_GetGyro(Gyro_Struct *gyro)
{
    uint8_t buff[6];
    Inf_MPU6050_ReadRegs(MPU_GYRO_XOUTH_REG, 6, buff);
    gyro->gyroX = (int16_t)(buff[0] << 8 | buff[1]);
    gyro->gyroY = (int16_t)(buff[2] << 8 | buff[3]);
    gyro->gyroZ = (int16_t)(buff[4] << 8 | buff[5]);
}

/**
 * @description: 获取陀螺仪的各轴的加速度
 * @param {Accel_Struct} *accel
 * @return {*}
 */
void Inf_MPU6050_GetAccel(Accel_Struct *accel)
{
    uint8_t buff[6];
    Inf_MPU6050_ReadRegs(MPU_ACCEL_XOUTH_REG, 6, buff);
    accel->accelX = (int16_t)(buff[0] << 8 | buff[1]);
```

```
    accel->accelY = (int16_t)(buff[2] << 8 | buff[3]);
    accel->accelZ = (int16_t)(buff[4] << 8 | buff[5]);
}

/**
 * @description: 获取陀螺仪各轴的加速度和角速度
 * @param {GyroAccel_Struct} *gyroAccel
 * @return {*}
 */
void Inf_MPU6050_GetGyroAccel(GyroAccel_Struct *gyroAccel)
{
    Inf_MPU6050_GetGyro(&gyroAccel->gyro);
    Inf_MPU6050_GetAccel(&gyroAccel->accel);

    if(isCalibrated) /* 判断是否校准过 */
    {
        /* 对值做校准 */
        gyroAccel->accel.accelX -= mpuOffset.accel.accelX;
        gyroAccel->accel.accelY -= mpuOffset.accel.accelY;
        gyroAccel->accel.accelZ -= mpuOffset.accel.accelZ;

        gyroAccel->gyro.gyroX -= mpuOffset.gyro.gyroX;
        gyroAccel->gyro.gyroY -= mpuOffset.gyro.gyroY;
        gyroAccel->gyro.gyroZ -= mpuOffset.gyro.gyroZ;
    }
}

/**
 * @description: 在陀螺仪静止条件下,对陀螺仪做校准
 * 在正常情况下, MPU6050 水平静止放置时:
 *     沿 x,y,z 各轴的角速度为 0
 *     沿 x,y 方向的加速度为 0
 *     沿 z 方向的加速是 16383
 * 校准的过程就是读出水平静止放置时的 6 个值,将来获取这个 6 个值时,需要减去这 6 个值
```

```
* @return {*}
*/

static void Inf_MPU6050_Calibrate(void)
{

    /* 1. 等待 MPU6050 处于静止状态
        a:判断沿着 x,y,z 的角速度本次和上次的差值小于 1 个阈值,就认为是静止状态
        为了减少误差,连续判断 100 次. 只判断角速度即可
        b:采样率设置的为 500Hz, 所以两次读取的间隔不能低于 2ms
    */

    uint16_t gyroThreshHold = 10; /* 静止阈值 */
    Gyro_Struct lastGyro;          /* 存储上次读取到的角速度 */
    Gyro_Struct currentGyro;       /* 存储这次读取到的角速度 */
    Inf_MPU6050_GetGyro(&lastGyro);
    uint8_t cnt = 100;
    debug_printfln("        陀螺仪: 准备校准");
    debug_printfln("        陀螺仪: 等待静止");
    while(cnt)
    {
        HAL_Delay(2);
        Inf_MPU6050_GetGyro(&currentGyro);
        if(abs(currentGyro.gyroX - lastGyro.gyroX) <= gyroThreshHold &&
            abs(currentGyro.gyroY - lastGyro.gyroY) <= gyroThreshHold &&
            abs(currentGyro.gyroZ - lastGyro.gyroZ) <= gyroThreshHold)
        {
            cnt--;
        }
        else
        {
            // cnt = 30; /* 一旦出现非静止状态, 把 cnt 的值重新初始化为 30 */
        }
        /* 把 current 值,赋值给 last, 方便下次使用*/
        lastGyro.gyroX = currentGyro.gyroX;
        lastGyro.gyroY = currentGyro.gyroY;
```

```
        lastGyro.gyroZ = currentGyro.gyroZ;
    }

    debug_printfln("        陀螺仪：已经静止");
    debug_printfln("        陀螺仪：开始校准");
    /* 2. 读出 6 个偏移量：多次读取，最后取平均值 */
    int32_t buff[6] = {0}; /* 存储 6 个量的和 */
    for(uint16_t i = 0; i < 256; i++)
    {
        Inf_MPU6050_GetGyroAccel(&mpuOffset);

        buff[0] += mpuOffset.gyro.gyroX - 0;
        buff[1] += mpuOffset.gyro.gyroY - 0;
        buff[2] += mpuOffset.gyro.gyroZ - 0;

        buff[3] += mpuOffset.accel.accelX - 0;
        buff[4] += mpuOffset.accel.accelY - 0;
        buff[5] += mpuOffset.accel.accelZ - 16383;

        HAL_Delay(3); /* 每 3ms 读取一次 */
    }

    mpuOffset.gyro.gyroX = (buff[0] >> 8); /* 除以 256 或者右移 8 位 */
    mpuOffset.gyro.gyroY = (buff[1] >> 8);
    mpuOffset.gyro.gyroZ = (buff[2] >> 8);

    mpuOffset.accel.accelX = (buff[3] >> 8);
    mpuOffset.accel.accelY = (buff[4] >> 8);
    mpuOffset.accel.accelZ = (buff[5] >> 8);

    isCalibrated = 1;
    debug_printfln("        陀螺仪：校准完成");
}
```

10.3 应用层

10.3.1 App_Flight.h

```
#ifndef __APP_FLIGHT_H
#define __APP_FLIGHT_H

#include "Common_Debug.h"
#include "Inf_Motor.h"
#include "Inf_MPU6050.h"
#include "Common_Filter.h"
#include "outputdata.h"
#include "math.h"
#include "Common_IMU.h"
#include "Common_PID.h"

extern EulerAngle_Struct eulerAngle;

void App_Flight_Start(void);

void App_Flight_GetMpuFilteredData(GyroAccel_Struct *gyroAccel);

void App_Flight_GetEulerAngle(GyroAccel_Struct *gyroAccel,
                              EulerAngle_Struct *eulerAngle,
                              float dt);

float App_Flight_GetNormAccZ(void);

void App_Flight_PIDPosture(GyroAccel_Struct *gyroAccel, EulerAngle_Struct
*eulerAngle, float dt);

void App_Flight_MotorControl(Com_Status isUnlocked);

void printGyroAccel(uint8_t *pre,
                    GyroAccel_Struct *gyroAccel);
void printfEulerAngle(EulerAngle_Struct *eulerAngle);

Com_Status App_Flight_RemoteUnlock(Com_Status isConnected);
```

```
void App_Flight_PidWithMoveDir(void);
```

```
#endif
```

10.3.2 App_Flight.c

```
#include "App_Flight.h"
```

```
/* 对 x 限幅度处理 */
```

```
#define LIMIT(x, min, max) ((x) < (min) ? (min) : ((x) > (max) ? (max) : (x)))
```

```
#define ABS(x) ((x) >= 0 ? (x) : (-(x)))
```

```
/*===== 6 个 姿态 PID 定义 开始=====*/
```

```
/* 外环：俯仰角度 内环：沿 Y 轴的角速度*/
```

```
static PID_Struct pitchPid = {.kp = -6.8f, .ki = 0.0f, .kd = 0.0f, .measure = 0.0f};
```

```
static PID_Struct gyroYPid = {.kp = 1.7f, .ki = 0.0f, .kd = 0.08f, .measure = 0.0f};
```

```
/* 外环：横滚角度 内环：沿 X 轴的角速度*/
```

```
static PID_Struct rollPid = {.kp = -6.8, .ki = 0, .kd = 0, .measure = 0};
```

```
static PID_Struct gyroXPid = {.kp = -1.7, .ki = 0, .kd = -0.08f, .measure = 0};
```

```
/* 外环：偏航角度 内环：沿 Z 轴的角速度*/
```

```
static PID_Struct yawPid = {.kp = -2.5, .ki = 0, .kd = 0, .measure = 0};
```

```
static PID_Struct gyroZPid = {.kp = -1, .ki = 0, .kd = 0, .measure = 0};
```

```
/*===== 6 个 姿态 PID 定义 结束=====*/
```

```
/* 欧拉角定义 */
```

```
EulerAngle_Struct eulerAngle;
```

```
/**
```

```
 * @description: 飞行模块启动
```

```
 * @return {*}
```

```
*/
```

```
void App_Flight_Start(void)
```

```
{

    debug_printfln("飞控模块：开始启动...");

    Inf_Motor_Init();

    Inf_MPU6050_Init();

    debug_printfln("飞控模块：完成启动...");

}

/**
 * @description: 获取滤波后的数据
 * @param {GyroAccel_Struct} gyroAccel
 * @return {*}
 */
void App_Flight_GetMpuFilteredData(GyroAccel_Struct *gyroAccel)
{
    /* 0. 获取 mpu 原始数据 */
    Inf_MPU6050_GetGyroAccel(gyroAccel);

    /* 1. 对角速度使用一阶低通滤波(噪声小) */
    static int32_t lastGyros[3] = {0}; /* 保存上一次的旧值 x y z */
    gyroAccel->gyro.gyroX      = Common_Filter_LowPass(gyroAccel->gyro.gyroX,
lastGyros[0]);
    gyroAccel->gyro.gyroY      = Common_Filter_LowPass(gyroAccel->gyro.gyroY,
lastGyros[1]);
    gyroAccel->gyro.gyroZ      = Common_Filter_LowPass(gyroAccel->gyro.gyroZ,
lastGyros[2]);

    lastGyros[0] = gyroAccel->gyro.gyroX; /* 把这次的保存，下次使用 */
    lastGyros[1] = gyroAccel->gyro.gyroY;
    lastGyros[2] = gyroAccel->gyro.gyroZ;

    /* 2. 对加速度使用简易版卡尔曼滤波(噪声大) */
    gyroAccel->accel.accelX = Common_Filter_KalmanFilter(&kfs[0],
gyroAccel->accel.accelX);
```



```
        gyroAccel->accel.accelY = Common_Filter_KalmanFilter(&kfs[1],
gyroAccel->accel.accelY);

        gyroAccel->accel.accelZ = Common_Filter_KalmanFilter(&kfs[2],
gyroAccel->accel.accelZ);
    }

/**
 * @description: 根据 mpu 的 6 轴数据, 获取表征姿态的欧拉角
 * @param {GyroAccel_Struct} *gyroAccel mpu 的 6 轴数据
 * @param {EulerAngle_Struct} *EulerAngle 计算后得到的欧拉角
 * @param {float} dt 采样周期 (单位 s)
 * @return {*}
 */
void App_Flight_GetEulerAngle(GyroAccel_Struct *gyroAccel,
                               EulerAngle_Struct *eulerAngle,
                               float dt)
{
    Common_IMU_GetEulerAngle(gyroAccel, eulerAngle, dt);
}

/**
 * @description: 获取 Z 轴上的加速度 (如果已经倾斜, 会考虑 z 轴上加速度的合成)
 * @return {*}
 */
float App_Flight_GetNormAccZ(void)
{
    return Common_IMU_GetNormAccZ();
}

/**
 * @description: 根据欧拉角和采样周期, 进行 PID 计算
 * @param {GyroAccel_Struct } *gyroAccel
 * @param {EulerAngle_Struct} *eulerAngle
 * @param {float } dt
 * @return {*}
 */
```

```
*/  
  
void App_Flight_PIDPosture(GyroAccel_Struct *gyroAccel,  
                           EulerAngle_Struct *eulerAngle,  
                           float dt)  
{  
    /* 俯仰 */  
    pitchPid.dt = dt;  
    pitchPid.measure = eulerAngle->pitch; /* 外 角度 */  
    gyroYPid.dt = dt;  
    gyroYPid.measure = gyroAccel->gyro.gyroY * Gyro_G; /* 内 角速度 */  
    Common_PID_Casecade(&pitchPid, &gyroYPid);  
  
    /* 横滚 */  
    rollPid.dt = dt;  
    rollPid.measure = eulerAngle->roll;  
    gyroXPid.dt = dt;  
    gyroXPid.measure = gyroAccel->gyro.gyroX * Gyro_G;  
    Common_PID_Casecade(&rollPid, &gyroXPid);  
  
    /* 偏航 */  
    yawPid.dt = dt;  
    yawPid.measure = eulerAngle->yaw;  
    gyroZPid.dt = dt;  
    gyroZPid.measure = gyroAccel->gyro.gyroZ * Gyro_G;  
    Common_PID_Casecade(&yawPid, &gyroZPid);  
}  
  
/**  
 * @description: 根据 PID 结果控制电机运动  
 * @return {*}  
 */  
  
void App_Flight_MotorControl(Com_Status isUnlocked)  
{  
  
    if(isUnlocked == Com_OK)
```

```

{
    /* 限制油门速度 */
    uint16_t speed;
    if(joyStick.isFixHeightPoint)
    {
        speed = LIMIT(joyStick.THRWithHeight, 0, 800);
    }
    else
    {
        speed = LIMIT(joyStick.THR, 0, 800);
    }

    /*
        1. 把 pid 的结果叠加到油门上
        俯仰: leftTop+rightTop      vs  leftBottom+rightBottom  符号相反    Y 轴角
速度
        横滚: leftTop+leftBottom    vs  rightTop+rightBottom    符号相反    X 轴角
速度
        偏航: leftTop+rightBottom   vs  leftBottom+rightTop     符号相反    Z 轴角
速度
    */
    leftTopMotor.speed = speed + gyroYPid.result + gyroXPid.result + gyroZPid.result;
    rightTopMotor.speed = speed + gyroYPid.result - gyroXPid.result - gyroZPid.result;
    leftBottomMotor.speed = speed - gyroYPid.result + gyroXPid.result - gyroZPid.result;
    rightBotomMotor.speed = speed - gyroYPid.result - gyroXPid.result + gyroZPid.result;

    /* 对叠加后的结果做限幅 */
    leftTopMotor.speed = LIMIT(leftTopMotor.speed, 0, 1000);
    rightTopMotor.speed = LIMIT(rightTopMotor.speed, 0, 1000);
    leftBottomMotor.speed = LIMIT(leftBottomMotor.speed, 0, 1000);
}

```

```
rightBotomMotor.speed = LIMIT(rightBotomMotor.speed, 0, 1000);

/* 为方便调试,当油门拉到最低时,所有马达速度置0 */
if(joyStick.THR <= 20)
{
    leftTopMotor.speed =
        rightTopMotor.speed =
        leftBottomMotor.speed =
        rightBotomMotor.speed = 0;
}
}
else
{
    /* 油门没有解锁 */
    leftTopMotor.speed =
        rightTopMotor.speed =
        leftBottomMotor.speed =
        rightBotomMotor.speed = 0;
}

/* 2. 让 motor 运动起来 */
Inf_Motor_AllWork();
}

/* =====用于测试打印的函数===== */
/**
 * @description: 打印 MPU6050 数据, 方便调试
 * @param {uint8_t} *pre 打印的前缀表示
 * @param {GyroAccel_Struct} *gyroAccel mpu6050 数据结构体
 * @return {*}
 */
void printGyroAccel(uint8_t *pre,
                    GyroAccel_Struct *gyroAccel)
{
```

```
printf("%s: gx=%d, gy=%d, gz=%d, ax=%d, ay=%d, az=%d\r\n",
    pre,
    gyroAccel->gyro.gyroX,
    gyroAccel->gyro.gyroY,
    gyroAccel->gyro.gyroZ,
    gyroAccel->accel.accelX,
    gyroAccel->accel.accelY,
    gyroAccel->accel.accelZ);
}

/**
 * @description: 输出欧拉角
 * @param {EulerAngle_Struct} eulerAngle
 * @return {*}
 */
void printfEulerAngle(EulerAngle_Struct *eulerAngle)
{
    printf("euler: pithc=%.2f, roll=%.2f, yaw=%.2f\r\n",
        eulerAngle->pitch,
        eulerAngle->roll,
        eulerAngle->yaw);
}

/**
 * @description: 检测遥控器是否发送解锁信号
 * @param {Com_Status} isConnected 遥控器是否连接成功
 *
 *          解锁逻辑：在油门最大值(1000)附近持续时间超过 1s,
 *          然后油门又拉到最低值(0)附近
 * @return {*} 是否解锁成功
 */
Com_Status App_Flight_RemoteUnlock(Com_Status isConnected)
{
    /*
        油门阶段划分：
    */
}
```

阶段 0: 油门初始.

在此阶段, 油门拉到最大值附近, 0->1

阶段 1: 油门处于最大值附近

在此阶段, 如果油门持续时间超过 1s, 则 1->2

否则, 1->0

阶段 2: 离开油门最大值附近

在此阶段, 如果油门回到最小值附近, 则 2->3

阶段 3: 进入油门最小值附近

在此阶段, 如果油门持续时间超过 1s, 则 3->4

阶段 4: 油门解锁阶段

另外:

如果当前处于阶段 4, 但是油门长时间处于最小值附近, 则锁定遥控器

然后重回阶段 0

```
*/  
  
static uint8_t stage      = 0;  
static uint16_t thrHighTime = 0; /* 油门最大值时间 */  
static uint16_t thrLowTime  = 0; /* 油门最小值时间 */  
  
/* 1. 如果 2.4g 失联, 返回解锁失败 */  
if(isConnected == Com_ERROR)  
{  
    stage      = 0;  
    thrHighTime = 0;  
    thrLowTime  = 0;  
    return Com_ERROR;  
}  
  
/* 2. 读取油门值 */  
uint16_t thr = joyStick.THR;
```

```
/* 如果油门值长时间处于最低值，则锁定 */

if(stage == 4 && thr <= 20) /* 如果目前是解锁阶段 并且油门低于 20 */
{
    static uint16_t lowDuration = 0; /* 记录最低值持续时间 */

    lowDuration++;

    if(lowDuration >= 250 * 60) /* 处于油门最小值超过 1m */
    {
        stage      = 0; /* 回到阶段 0 */

        lowDuration = 0; /* 最低值持续时间清零 */

        thrLowTime  = 0;

        thrHighTime = 0;

        return Com_ERROR; /* 返回锁定 */

    }
}

switch(stage)
{
    case 0:
    {
        thrHighTime = 0;

        thrLowTime  = 0;

        if(thr >= 980)
        {
            stage = 1;
        }

        break;
    }

    case 1:
    {
        if(thr >= 980)
        {
            thrHighTime++;

            if(thrHighTime >= 250)
            {

```

```
        stage = 2;
    }
}
else
{
    thrHighTime = 0;
    thrLowTime  = 0;
    stage       = 0;
}

break;
}
case 2:
{
    if(thr <= 20)
    {
        stage = 3;
    }
    break;
}

case 3:
{
    if(thr <= 20)
    {
        thrLowTime++;
        if(thrLowTime >= 250)
        {
            stage = 4;
        }
    }
    else
    {
        thrLowTime = 0;
    }
}
```



```

        thrHighTime = 0;

        stage      = 0;

    }

    break;

}

case 4:

{

    return Com_OK;

}

default:

    break;

}

return Com_ERROR;

}

/**
 * @description: 根据摇杆控制电机运动方向，就是把要摇杆值作用在角度的期望值
 * @return {*}
 */
void App_Flight_PidWithMoveDir(void)
{

    /* 我们的值是 0-1000 减去 500 之后,变成-500 到+500 */
    pitchPid.desire = (joyStick.PIT - 500) * 0.03;
    rollPid.desire  = (joyStick.ROL - 500) * 0.03;
    yawPid.desire   = (joyStick.YAW - 500) * 0.03;

}

```

10.4 任务调度

```

/* 2. 飞行控制任务 */
void      flightControlTask(void *arg);
TaskHandle_t flightControlHandle;

```

```
#define FLIGHT_CONTROL_PRIORITY 8

#define FILGHT_CONTORL_EXECUTE_CYCLE 4

/*
    2. 飞行控制任务函数
*/

void flightControlTask(void *arg)
{
    /* 启动通讯模块：启动的过程中，不要被打断 */
    debug_printfln("飞控任务：开始调度");

    float dt = FILGHT_CONTORL_EXECUTE_CYCLE / 1000.0; /* 采样周期 0.004s */
    /* 用来计算绝对延时时间 */
    TickType_t pxPreviousWakeTime = xTaskGetTickCount();
    while(1)
    {
        /* 1. 判断遥控器是否已经解锁 */
        isUnlocked = App_Flight_RemoteUnlock((Com_Status)isConnected);

        if(isUnlocked == Com_OK)
        {
            /* 2. 获取滤波后的 MPU6050 的 6 轴数据 */
            portENTER_CRITICAL();
            App_Flight_GetMpuFilteredData(&gyroAccel);
            portEXIT_CRITICAL();

            /* 3. 根据 6 轴数据得到欧拉角 */
            App_Flight_GetEulerAngle(&gyroAccel, &eulerAngle, dt);

            /* 4. 根据欧拉角和 6 轴数据，进行 PID 计算 */
            App_Flight_PIDPosture(&gyroAccel, &eulerAngle, dt);

            /* 5. 根据摇杆的方向，设置外环角度的期望值 */
            App_Flight_PidWithMoveDir();
        }
    }
}
```

```
/* 9. 把 PID 计算的结果作用到电机上,控制电机运动 */  
App_Flight_MotorControl(isUnlocked);  
  
xTaskDelayUntil(&pxPreviousWakeTime, FILGHT_CONTORL_EXECUTE_CYCLE);  
}  
}
```

第 11 章 main 函数

```
#include "main.h"  
#include "i2c.h"  
#include "spi.h"  
#include "tim.h"  
#include "usart.h"  
#include "gpio.h"  
  
#include "Common_Debug.h"  
#include "Common_Delay.h"  
#include "Common_Config.h"  
#include "App_Task.h"  
#include "Inf_IP5305T.h"  
#include "Inf_MPU6050.h"  
void SystemClock_Config(void);  
int main(void)  
{  
    HAL_Init();  
  
    SystemClock_Config();  
  
    MX_GPIO_Init();  
    MX_USART2_UART_Init();  
    MX_TIM1_Init();  
    MX_TIM2_Init();  
    MX_TIM3_Init();
```

```
MX_TIM4_Init();
MX_I2C1_Init();
MX_SPI1_Init();
MX_I2C2_Init();
/* 1. 初始化 debug 模块 */
debug_init();
debug_printfln("无人机项目之 --- 飞控程序....");
HAL_Delay(3000);
/* 2. 各个模块的初始化 */
/* 2.1 飞控模块启动 */
App_Flight_Start();

/* 2.2 通讯模块启动 */
App_Communication_Start();

/* 3. 启动 FreeRTOS */
debug_printfln("开始启动 FreeRTOS...");
App_FreeRTOS_Start();
while(1)
{
}
}
```